

INHALTSVERZEICHNIS

I Thesis

1	Erweiterung der Treemap-Visualisierung	2
1.1	Kriterien für Treemap Layouts	2
1.2	Auswahl der Projekte	4
1.3	Analyse von Softwareprojekten	7
1.3.1	Ordnerketten	8
1.4	Abstände zwischen Knoten	9
1.5	Angestrebtes Seitenverhältnis	11
1.6	Reihenfolge der Knoten im ersten Layoutschritt	13
1.7	Zweifache Berechnung	16
1.7.1	Größenanpassung	22
1.8	Platzierung der Knoten nach erstem Layoutschritt	30
1.9	Mehrfache Berechnung	35
1.9.1	Iterative Abstandsvergrößerung	36
1.10	Optimierung von Ordnerketten	36
1.11	Abstände zwischen Geschwister Knoten	41
1.12	Beschriftung der Knoten	41
1.13	Fazit	45
2	Umfrage	47
2.1	Ablauf der Umfrage	47
2.2	Ergebnisse der Umfrage	51
2.2.1	Bewertung anhand von Szenarien	51
2.2.2	Direkte Bewertung der Visualisierungen	53
2.2.3	Zusätzliche Anmerkungen der Befragten	54
2.3	Fazit	55
3	Fazit	57

II Appendix

A	Dateien	60
	Literatur	75

ABBILDUNGSVERZEICHNIS

Abbildung 1.1	Größenverteilung roh	5
Abbildung 1.2	Tiefe nach Sprache	6
Abbildung 1.3	Breite nach Sprache	7
Abbildung 1.4	Größe vs. Tiefe	8
Abbildung 1.5	Größe vs. Breite	9
Abbildung 1.6	Mitwirkende vs. Kinderzahl	10
Abbildung 1.7	Mitwirkende vs. Tiefe	11
Abbildung 1.8	Größenverteilung ausgewählt	12
Abbildung 1.9	Kinder pro Ordner	13
Abbildung 1.10	Verteilung der Tiefe	13
Abbildung 1.11	Kettentiefe nach Sprache	14
Abbildung 1.12	Verteilung Kettenlängen	15
Abbildung 1.13	Ordneranteil in Ketten	16
Abbildung 1.14	Absoluter Abstand zu Relativer Abstand	17
Abbildung 1.15	React mit relativem Abstand von 0,4	17
Abbildung 1.16	Qwen3 mit relativem Abstand von 4	18
Abbildung 1.17	Qwen3 mit relativem Abstand von 1	18
Abbildung 1.18	Qwen3 mit relativem Abstand von 35	19
Abbildung 1.19	Zusammenhang zwischen dem angestrebten Seitenverhältnis und dem tatsächlich erreichten Seitenverhältnis bei der Anwendung des Standard d3.js Squarify Algorithmus auf die ausgewählten Repositories.	19
Abbildung 1.20	Fehlende Knoten über relativen Abstand	20
Abbildung 1.21	Flächenverhältnis über relativen Abstand	20
Abbildung 1.22	Seitenverhältnis über relativen Abstand	20
Abbildung 1.23	Platznutzung über relativen Abstand	21
Abbildung 1.24	Zeitaufwand über relativen Abstand	21
Abbildung 1.25	Treemap Layout generiert mit dem Squarify Algorithmus nach Abschnitt ?? mit einem Abstand von 0 und der einfachen Größenanpassung auf der händisch erstellen map (siehe Anhang).	23
Abbildung 1.26	Treemap Layout generiert mit dem Squarify Algorithmus nach Abschnitt ?? mit einem Abstand von 1 und der absoluten Größenanpassung auf der händisch erstellen map (siehe Anhang).	24
Abbildung 1.27	Treemap Layout generiert mit dem Squarify Algorithmus nach Abschnitt ?? mit einem Abstand von 1 und der relativen Größenanpassung auf der händisch erstellen map (siehe Anhang).	26

Abbildung 1.28	Treemap Layout generiert mit dem Squarify Algorithmus nach Abschnitt ?? mit einem Abstand von 1 und der einfachen Größenanpassung auf der händisch erstellen map (siehe Anhang) und der Skalierung der Knoten.	27
Abbildung 1.29	Treemap Layout generiert mit dem Squarify Algorithmus nach Abschnitt ?? mit einem Abstand von 1 und der relativen Größenanpassung auf der händisch erstellen map (siehe Anhang) und der Skalierung der Knoten.	28
Abbildung 1.30	Fehlende Knoten über relativen Abstand	28
Abbildung 1.31	Flächenverhältnis über relativen Abstand	29
Abbildung 1.32	Seitenverhältnis über relativen Abstand	29
Abbildung 1.33	Platznutzung über relativen Abstand	29
Abbildung 1.34	Zeitaufwand über relativen Abstand	30
Abbildung 1.35	Zeitaufwand über relativen Abstand	31
Abbildung 1.36	Fehlende Knoten über relativen Abstand	32
Abbildung 1.37	Fehlende Knoten über relativen Abstand	33
Abbildung 1.38	Flächenverhältnis über relativen Abstand	33
Abbildung 1.39	Seitenverhältnis über relativen Abstand	34
Abbildung 1.40	Platznutzung über relativen Abstand	34
Abbildung 1.41	Zeitaufwand über relativen Abstand	34
Abbildung 1.42	Fehlende Knoten über relativen Abstand	36
Abbildung 1.43	Fehlende Knoten über relativen Abstand	37
Abbildung 1.44	Flächenverhältnis über relativen Abstand	37
Abbildung 1.45	Fehlende Knoten über relativen Abstand	39
Abbildung 1.46	Flächenverhältnis über relativen Abstand	39
Abbildung 1.47	Seitenverhältnis über relativen Abstand	39
Abbildung 1.48	Platznutzung über relativen Abstand	40
Abbildung 1.49	Zeitaufwand über relativen Abstand	40
Abbildung 1.50	Fehlende Knoten über relativen Abstand	42
Abbildung 1.51	Flächenverhältnis über relativen Abstand	42
Abbildung 1.52	Seitenverhältnis über relativen Abstand	43
Abbildung 1.53	Platznutzung über relativen Abstand	43
Abbildung 1.54	Zeitaufwand über relativen Abstand	44
Abbildung 2.1	Beispiel Visualisierung der Umfrage	50
Abbildung 2.2	Bewertungen pro Szenario	52
Abbildung 2.3	Bewertung nach Zielgruppe	53

TABELLENVERZEICHNIS

Tabelle 1.1	Kennzahlen Repositories	8
Tabelle 1.2	Durchschnittliches und median Seitenverhältnis bei verschiedenen angestrebten Seitenverhältnissen um 1 herum bei der Anwendung des Standard Squarify Al- gorithmus auf die ausgewählten Repositories.	15
Tabelle 2.1	Passend vs. nicht passend	51
Tabelle 2.2	Bewertungen der Visualisierungsexperten	54
Tabelle 2.3	Platzierung im Vergleich	55

LISTINGS

Listing A.1	Händisch erzeugte map	60
Listing A.2	Händisch erzeugte map 2	62
Listing A.3	Die zur Analyse ausgewählten GitHub-Repositories . .	65

Teil I

THESIS

Da die Treemap-Visualisierung die weit verbreitetste Visualisierungsmethode für die Darstellung von Software-Qualitätsmetriken ist und in der Literaturrecherche (siehe Abschnitt ??) zudem sehr gut abgeschnitten hat, macht es Sinn speziellen Augenmerk auf diese Visualisierung zu legen. Zudem wurde in der Auswertung der Literaturrecherche gefolgert, dass die Qualität der Treemap-Visualisierung stark schwanken kann und je nach Anwendungsfall optimiert werden sollte. Natürlich gibt es auch andere Treemap-Algorithmen, wie zum Beispiel den Slice-and-Dice Algorithmus [shneiderman1992tree] oder den Pivot-by-Middle Algorithmus [SWo1]. Der Squarify Algorithmus zeigt aber in der Literatur die besten Ergebnisse in Bezug auf das Seitenverhältnis der Knoten [SWo1]. Außerdem ist der Squarify Algorithmus [BHVWoo], wie er in Abschnitt ?? beschrieben wurde, ist für das erstellen von Treemap-Layouts ein sehr vielversprechender Algorithmus, der von vielen Tools und Visualisierungen, die wir zuvor vorgestellt haben verwendet wird [Gmbb; Gmba; WLo7]. Wir werden diesen Algorithmus deshalb auf verschiedene Weisen erweitert und anpassen, um das Layout für unseren Anwendungsfall zu verbessern. Dabei wird das Layoutproblem, wie es in Abschnitt ?? beschrieben wurde, eine zentrale Rolle spielen, da dieses ein großes Problem für die Treemap-Visualisierung darstellt. Zudem werden aber am Anfang des Kapitels noch weitere Kriterien aufgestellt, die für die Qualität der Treemap-Visualisierung relevant sind und die in diesem Kapitel optimiert werden sollen. Nach dem Aufstellen der Kriterien werden wir eine Datenanalyse von OpenSource Projekten durchführen, um mehr über typische Strukturen von Software zu lernen. Zudem Dient die Datenanalyse dazu, ein Set an typischen, breit gefächerten und repräsentativen Software-Projekten zu auswählen, die für die Evaluation in diesem Kapitel verwendet werden. Anschließend werden die verschiedenen Erweiterungen und Anpassungen des Squarify Algorithmus beschrieben und evaluiert, um am Ende einen Vorschlag für ein verbessertes Treemap-Layout zu machen, das für unseren Anwendungsfall besonders gut geeignet ist.

1.1 Kriterien für Treemap Layouts

Um diese fünf Aspekte zu erreichen, definieren wir sechs Kriterien für das 2D layout, die diese Aspekte messbar machen. Diese Kriterien sollen helfen, die Qualität der Visualisierung zu bewerten und zu vergleichen. Die Kriterien sind:

- **Knoten sichtbarkeit:** Es sollte keine Knoten geben, die aufgrund von Abständen oder anderen Gründen nicht sichtbar sind. Dieses Ziel spielt speziell auf das in abschnitt ?? beschriebene Problem ab. "Missing no-

des are not addressed in the existing treemap literature. "[LF08, S. 5] Diesen Wert kann man einfach zählen, indem Alle Knoten gezählt werden, deren Länge oder Breite kleiner oder gleich 0 ist. Dieses Kriterium betrachten wir von allen Kriterien als das wichtigste, da für unseren Anwendungsfall die Sichtbarkeit aller Knoten essenziell ist. Dies ist vor allem dann der Fall, wenn das 2D Layout in 2.5D extrudiert wird. Selbst wenn ein Knoten eine sehr kleine Fläche hat, kann es sein, dass dieser Knoten eine sehr hohe Höhe oder eine schlechte Farbe hat, wodurch es trotzdem wichtig ist, diesen Knoten zu sehen.

- **Flächengröße:** Um die Korrelation mit dem Code zu gewährleisten, sollte die Fläche der Knoten proportional zum Metrikwert sein. Wir berechnen dies einfach indem wir die Fläche des Knotens durch den Metrikwert teilen. Im Optimalfall sollte dieser Wert für alle Knoten gleich sein, um eine perfekte Proportionalität zu gewährleisten. Wir messen also die Abweichung des Wert-Fläche-Verhältnisses vom Mittelwert. Um diese Werte besser einordnen und vergleichen zu können, werden wir den Varianzkoeffizienten verwenden. Dies betrachten wir als das zweitwichtigste Kriterium, da die Fläche der Knoten eine wichtige Visuelleigenschaft ist, die möglichst genau den Metrikwerten entsprechen sollte.
- **Seitenverhältnis:** Um die visuelle Komplexität zu reduzieren und die Verständlichkeit zu erhöhen, sollte das Seitenverhältnis der Knoten möglichst nahe bei 1:1 liegen. Dies verbessert die Lesbarkeit der Knoten und macht es einfacher, die Informationen zu erfassen. Wir teilen einfach die größere Seitenlänge durch die kleinere Seitenlänge, um das Seitenverhältnis zu berechnen. Ein Wert von 1 bedeutet also ein perfektes Quadrat, während höhere Werte auf längliche Rechtecke hinweisen.
- **Platznutzung:** Es sollte so wenig Fläche wie möglich ohne Informationsgehalt bleiben. Als Gegenbeispiel kann man die Order-Knoten sehen, wie sie in der CodeCity Arbeit beschrieben wurden, bei denen die Fläche der Knoten nicht proportional zur Anzahl der Zeilen im Code ist, wodurch die Fläche an sich keinen Informationsgehalt mehr hat und außerdem viel leere Fläche entsteht. Wir messen das, indem wir die Fläche aller Blatt-Knoten durch die Fläche des Wurzelknotens teilen. Ein Wert von 1 bedeutet also, dass die gesamte Fläche des Wurzelknotens von den Blatt-Knoten eingenommen wird.
- **Zeitaufwand:** Die Generierung des Layouts sollte in einem angemessenen Zeitrahmen erfolgen, um eine schnelle Visualisierung zu ermöglichen. Dies verbessert die Skalierbarkeit und generelle Nutzbarkeit der Visualisierung. Anzumerken ist, dass alle Berechnungen auf einem M2 MacBook durchgeführt wurden, und alles in TypeScript implementiert wurde. Das ist auch praxisnah, da viele Tools in JavaScript/TypeScript implementiert sind und bei Nutzern lokal gerendert werden [Gmbb; Gmba; Cod]. Anzumerken ist, dass die gemessene Zeit nur die Aus-

führung des Layout-Algorithmus beinhaltet und nicht die Zeit für das Rendering der Visualisierung.

“Especially for the node weights, we assess if the algorithms use the weight values to scale the sizes of leaf nodes. Typically, the size of parent nodes is adjusted to the spatial consumption of the child nodes.”[SLD20, S. 3] - Scheibel et al. sagen also, für sie ist es nicht so wichtig, dass die Größe der Elternknoten proportional zu den Metrikwerten der Knoten ist.

1.2 Auswahl der Projekte

In diesem Abschnitt wird eine repräsentative Auswahl von Softwareprojekten beschrieben, die für die Analyse von Softwarestrukturen und die Evaluation der Erweiterung des Squarify-Algorithmus verwendet wird.

Als Quelle dient GitHub, da dort eine große Zahl an Open-Source-Projekten vorliegt und die Daten gut zugänglich sind. Es wird nicht von signifikanten Unterschieden zwischen Open-Source- und Closed-Source-Projekten ausgegangen [Rag+05; PSEo4]. Die Projektauswahl wird breit gefächert angelegt, um ein möglichst breites Spektrum abzudecken und spätere Aussagen zur Struktur von Softwareprojekten allgemeingültiger zu machen. Eine reine Zufallsauswahl von GitHub würde voraussichtlich zu einer Verzerrung zugunsten kleinerer, weniger bekannter Projekte mit häufig verwendeten Programmiersprachen führen. Abbildung 1.1 zeigt die Größenverteilung von 2500 zufällig ausgewählten Repositories. Dabei zeigt sich, dass kleinere Projekte deutlich häufiger sind. Zum Größenvergleich gilt das React-Repository als sehr groß und liegt bei etwa 1 GB. Daher werden Kriterien herangezogen, die die Struktur von Softwareprojekten beeinflussen könnten.

Da keine etablierten Quellen eine geeignete Kriterienliste vorgeben, erfolgt die Ableitung heuristisch aus den über die GitHub-API verfügbaren Metadaten. Die Relevanz der in Betracht gezogenen Kriterien wird empirisch geprüft. Ein Anspruch auf Vollständigkeit besteht nicht, was für die Ziele dieses Abschnitts jedoch nicht erforderlich ist.

Im Folgenden werden zunächst potenziell relevante Kriterien hinsichtlich ihres Einflusses auf die Struktur bewertet. Anschließend wird anhand der ausgewählten Kriterien die konkrete Projektauswahl beschrieben.

Programmier Sprache

Es werden Projekte in verschiedenen Programmiersprachen betrachtet. Unterschiede in Typisierung, Paradigmen und Ökosystemen können die Projektstruktur beeinflussen. Die Sprachen werden nach Popularität und Paradigma ausgewählt:

- **Python:** Zweitpopulärste Sprache [Sof], dynamisch typisiert, imperativ und objektorientiert.
- **JavaScript:** Populärste Sprache [Sof], dynamisch typisiert, Struktur häufig durch Frameworks geprägt.

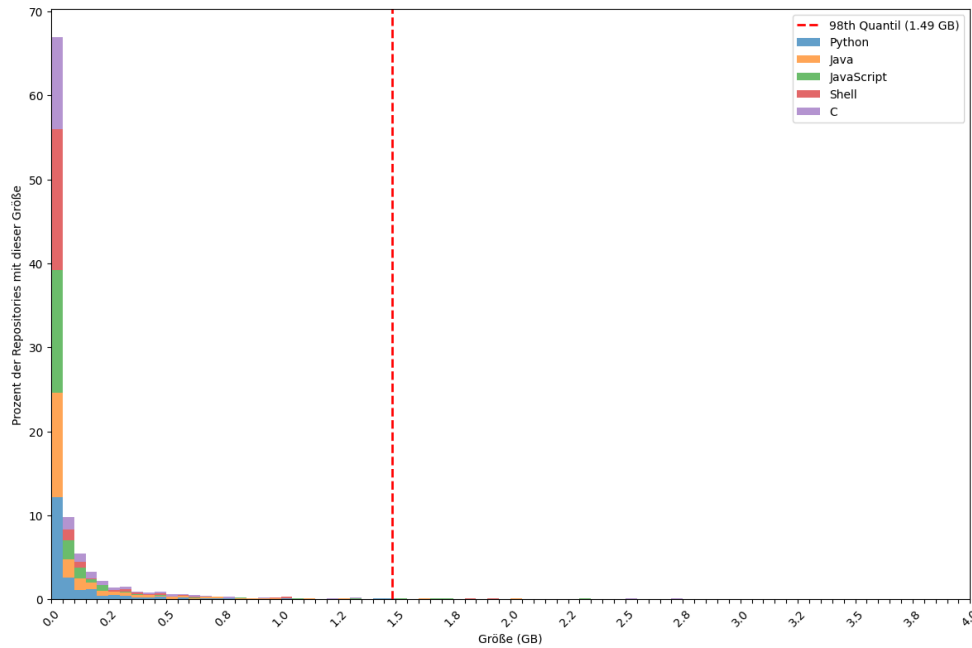


Abbildung 1.1: Größenverteilung zufällig gewählter Repositories

- **Java:** Drittpopulärste Sprache [Sof], statisch typisiert, objektorientiert, klare Konventionen.
- **C:** Etabliert und weit verbreitet (Rang 9 [Sof]), häufig hardwarenahe Projekte mit abweichenden Strukturmustern.
- **Shell:** Skriptsprache (Rang 8 [Sof]), häufig für Automatisierung und Administration.

Die Abbildungen 1.2 und 1.3 verdeutlichen einen Einfluss der Programmiersprache auf die Struktur. Daher wird die Sprache als Auswahlkriterium berücksichtigt.

Projektgröße

Die Anzahl an Dateien ist offensichtlich ein zentraler Faktor für die Struktur, etwa für Knotenzahl und Hierarchietiefe. GitHub erlaubt jedoch nur eine Filterung nach Größe in Bytes, nicht nach Anzahl der Dateien. Zur Plausibilisierung wird geprüft, inwieweit die Größe mit strukturellen Kennzahlen korreliert.

Beide Abbildungen zeigen einen geringen Einfluss der Größe auf strukturelle Eigenschaften. Dies rechtfertigt die Berücksichtigung der Größe als Auswahlkriterium, auch weil ansonsten keine praktikable Möglichkeit besteht, die Knotenzahl vorab einzuschätzen, da die GitHub-API diese Information nicht bereitstellt.

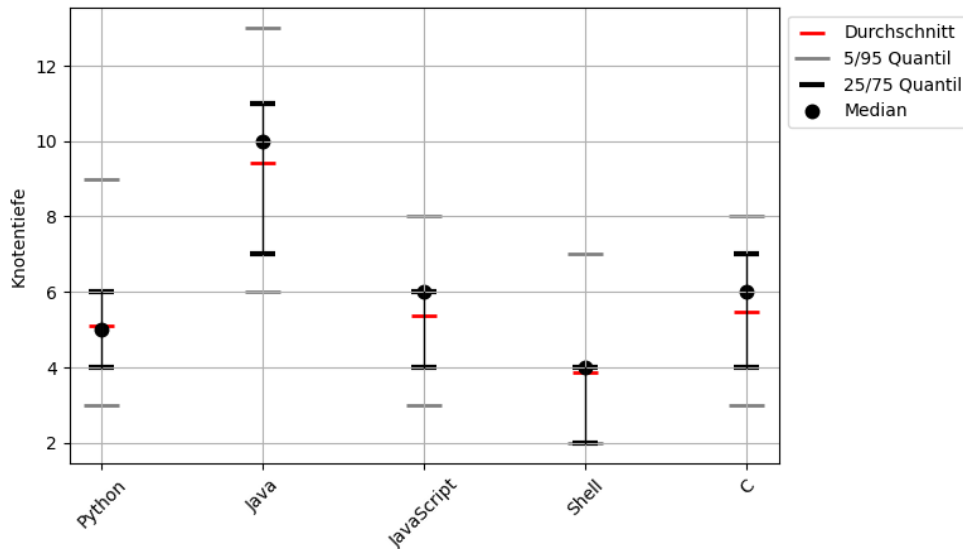


Abbildung 1.2: Die Tiefe variiert deutlich nach Sprache. In Java liegen die meisten Knoten (95%) deutlich tiefer (Tiefe 7) als in Shell mit 75% bei Tiefe 4.

Ungenutzte Kategorien

Weitere Kategorien wie Popularität in Form von Sternen oder Anzahl der Mitwirkenden könnten strukturelle Effekte haben. Eine Vorprüfung zeigt jedoch keine tragfähige Relevanz für diese Analyse:

1. Popularität korreliert nicht zuverlässig mit Softwarequalität [Saj+14]. Zudem sind viele Projekte mit hoher Sternzahl keine Softwareprojekte im engeren Sinn, etwa Sammlungen von Ressourcen [Eva]. Die Popularität wird daher nicht berücksichtigt.

2. Die Anzahl der Mitwirkenden kann die Qualität und Struktur beeinflussen [CAC20]. Projekte mit nur einer Person werden ausgeschlossen, da sie oft nicht repräsentativ sind. Für die Gruppe mit mindestens zwei Mitwirkenden zeigen die Daten jedoch keinen erkennbaren Einfluss der Mitwirkendenzahl auf Tiefe oder Breite der Struktur, wie in den Abbildungen 1.6 und 1.7 dargestellt.

Für die Auswahl werden folglich die Kriterien Sprache und Größe kombiniert. Fünf Sprachen werden betrachtet, und die Größenverteilung wird so abgedeckt, dass alle Kombinationen hinreichend repräsentiert sind, ohne den rechnerischen Rahmen zu sprengen.

Zudem werden Projekte wie *The most comprehensive database of Chinese poetry* oder *A list of funny and tricky JavaScript examples* manuell ausgeschlossen, da es sich nicht um Softwareapplikationen im engeren Sinne handelt. Dadurch ist sichergestellt, dass die ausgewählten Repositories tatsächliche Softwareprojekte sind.

Ziel ist eine Abdeckung bis zum 98. Größenquantil von etwa 1,5 GB, um auch große Projekte einzubeziehen. Oberhalb dieser Größe wird eine sinnvolle Visualisierung praktisch schwierig, da die Knotenzahl sehr groß wird.

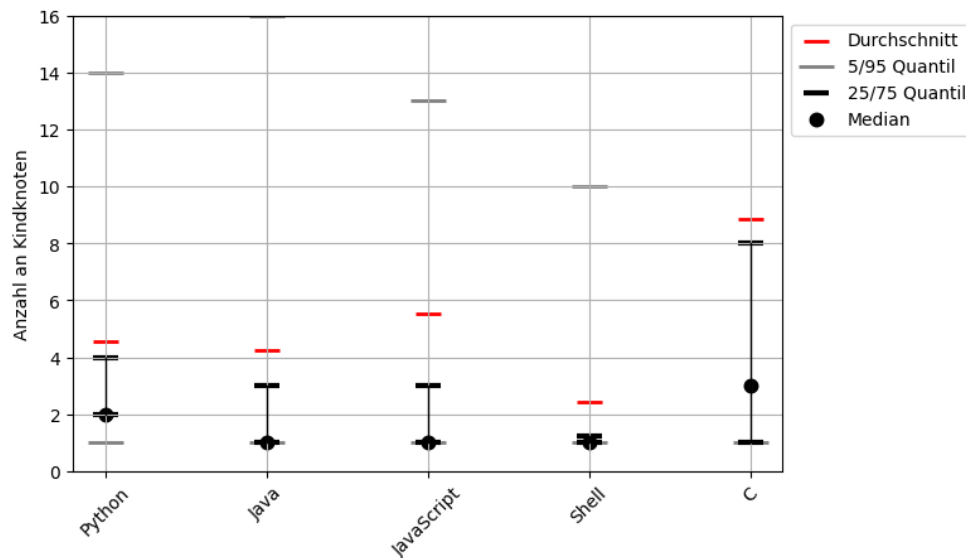


Abbildung 1.3: Die Breite variiert im Mittel zwischen den Sprachen, die Quantilsintervalle überlappen jedoch weitgehend.

Die Auswahl erfolgt größenstratifiziert bis 1,5 GB mit etwa linearer Abdeckung, also etwa alle 0,1 GB ein Projekt. Dies führt zu genau 15 Projekten pro Sprache, insgesamt 70 Projekten. Die konkrete Liste findet sich im Anhang, siehe Listing A.3.

Abbildung 1.8 zeigt die Größenverteilung der ausgewählten Repositories je Sprache. Die Verteilung ist bis 1,5 GB gleichmäßig abgedeckt.

1.3 Analyse von Softwareprojekten

Die Struktur- und Metrikdaten werden durch Klonen der Repositories und Analyse mit dem CodeCharta Analysis CLI gewonnen [Gmbd]. Dieses Tool analysiert die Struktur der Repositories und berechnet für jede Datei die Metriken, die später für die Visualisierung verwendet werden. Es werden nur Dateien berücksichtigt, die zur entsprechenden Programmiersprache gehören, etwa .py für Python. Genutzt wird der unified parser mit folgendem Befehl:

```
ccsh unifiedparser "$name" -o "${REPOS_DIR}/${name}.cc.json" -nc
  "--verbose=false" "--exclude=" "--file-extensions="
```

Das Ergebnis ist eine JSON-Datei im in Abschnitt ?? definierten Format. Ein Beispiel findet sich im Anhang (Listing A.1).

Im Folgenden werden Grenzen und typische Eigenschaften hierarchischer Strukturen aus dateibasierten Softwarehierarchien aufgezeigt, um Algorithmen und Visualisierungen dieser Arbeit gezielt auf diese Strukturen zu optimieren und Laufzeiten realistisch einzuschätzen.

Abbildung 1.9 zeigt, dass die meisten Ordner nur ein Kind besitzen, was im nächsten Abschnitt relevant wird (siehe Abschnitt 1.3.1). Das 95%-Quantil liegt bei 18. An diesem Wert orientieren sich die folgenden Abschnitte.

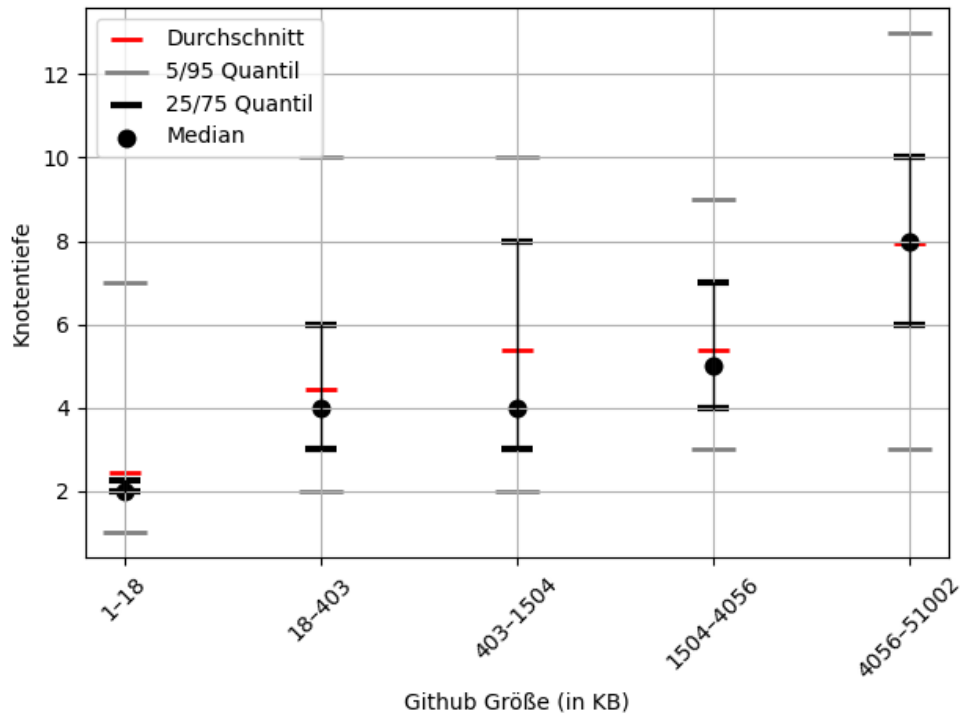


Abbildung 1.4: Die Hierarchietiefe korreliert positiv mit der Repository-Größe (Pearson $r = 0,37$). Die Korrelation ist schwach und weitere Faktoren wie die Sprache spielen eine Rolle.

	Durchschnitt	Median	Standardabweichung
Dateianzahl	3262	708	7391
Ordneranzahl	958	242	2089
Knotenzahl	4221	1108	9308

Tabelle 1.1: Kennzahlen über alle analysierten Repositories, gerundet auf ganze Zahlen.

Abbildung 1.10 zeigt die Verteilung der Tiefe aller Knoten. Die meisten Knoten liegen in mittleren Tiefen von 3 bis 11. Sehr tiefe Knoten sind selten. Das 95%-Quantil liegt bei Tiefe 13, was in den folgenden Abschnitten als Orientierung dient.

1.3.1 Ordnerketten

Es liegt die Vermutung nahe, dass die hohe Tiefe insbesondere in Java aus stark verschachtelten Paketstrukturen resultiert. Abbildung 1.11 zeigt die Kettentiefe nach Sprache. In Java ist die Varianz hoch und der Mittelwert liegt über den übrigen Sprachen. Ein Beispiel ist das Open-Source-Projekt CodeCharta [Gmbb]: `codecharta/analysis/analysers/AnalyserInterface/src/main/kotlin/de/maibornwolff/codecharta/analysers/analyserinterface/AnalyserInterface.kt`. Die Kette verläuft ab dem Ordner *main* bis *analyser-interface*.

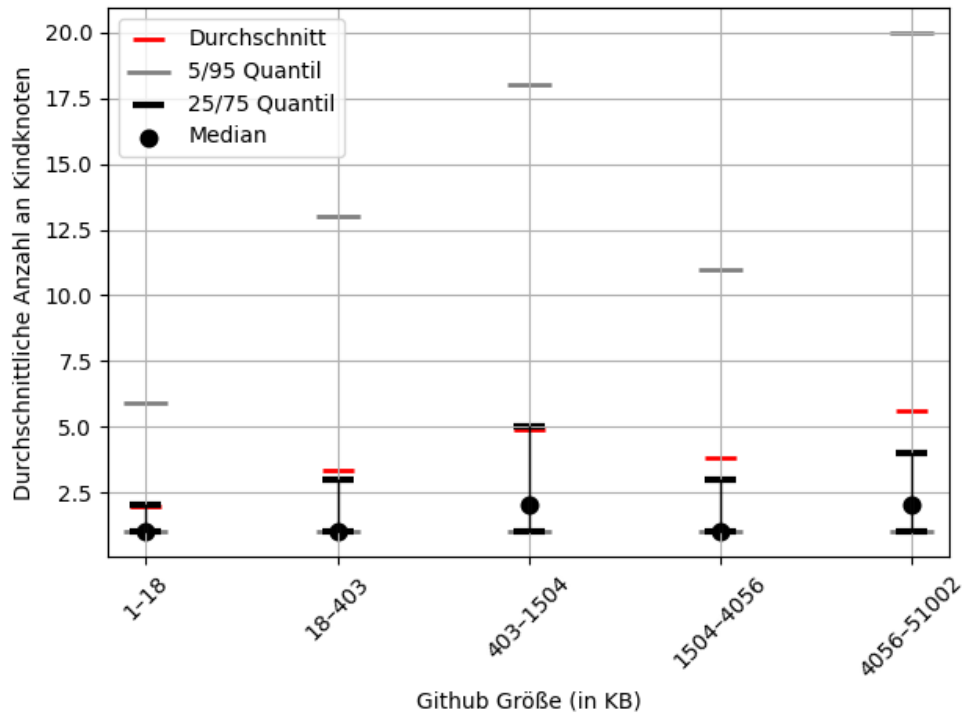


Abbildung 1.5: Die durchschnittliche Kinderzahl pro Knoten korreliert schwach positiv mit der Repository-Größe (Pearson $r = 0,09$).

Für die Verteilung der Kettenlängen zeigt Abbildung 1.12, dass die meisten Ketten nur eine Ordnerstufe umfassen, also Ordner zu Ordner zu Datei oder weiteren Ordnern. Längere Ketten, etwa der Länge 4, treten jedoch ebenfalls häufig auf. Abbildung 1.13 zeigt den Anteil der Ordner, die Teil einer Kette sind. Über alle Sprachen liegt der Mittelwert bei etwa 33%, in Java bei etwa 50%.

Die Abbildungen 1.12 und 1.13 machen sichtbar, dass viele Ketten existieren, insbesondere in Java, und dass ein großer Anteil der Ordner Teil solcher Ketten ist. Zudem können Ketten nicht nur eine Ordnerstufe tief sein, sondern häufig auch vier Ordnerstufen. Diese Erkenntnis wird in Abschnitt 1.10 eine Rolle spielen.

1.4 Abstände zwischen Knoten

Wie zuvor beschrieben, entsteht das Treemap-Problem erst durch das Hinzufügen von Abständen zwischen den Knoten (siehe Abschnitt ??). Je Größer die Abstände sind, desto größer wird das Problem.

Dabei ist die Wahl des Abstands erstmal subjektiv und kommt auf die Software an und darauf, was man darstellen will. Will man den Fokus auf Struktur oder auf Metriken legen?

Nesting offsets give users control over the allocation of display space between internal and leaf nodes. Larger offsets put grea-

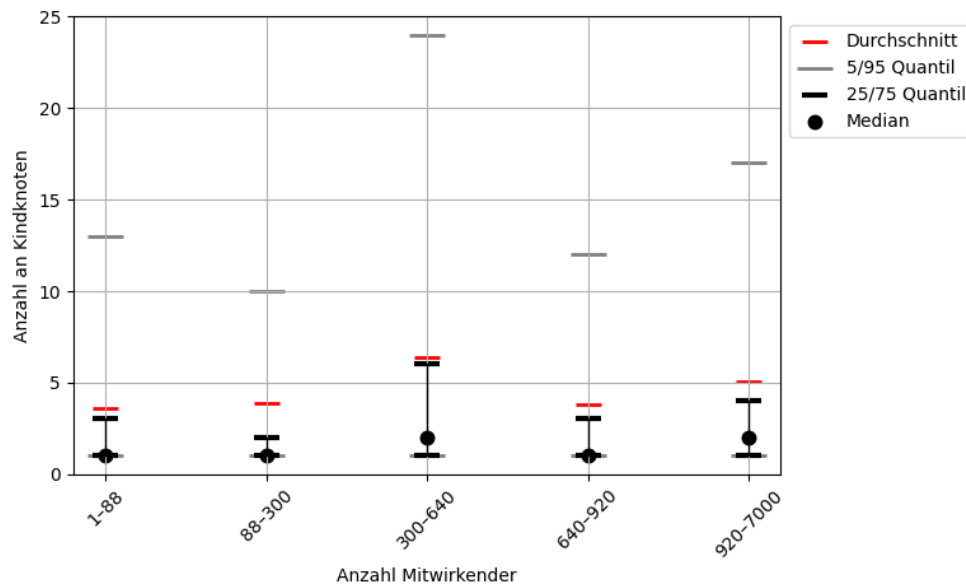


Abbildung 1.6: Die Anzahl der Mitwirkenden zeigt keinen erkennbaren Einfluss auf die durchschnittliche Kinderzahl eines Knotens.

ter emphasis on internal nodes and hence the structure of the hierarchy; smaller offsets emphasize leaf nodes. [Joh93, S. 48]

Alle Messwerte und Ergebnisse der Evaluation hängen dabei natürlich sowohl von den Analysierten Daten als auch von der Wahl der Größe des Abstands ab. Je nachdem wie groß der Abstand gewählt wird, desto größer wird das Treemap Problem, man will aber tendenziell nicht zu kleine Abstände, damit man besser die Struktur der Software erkennen kann. Aus diesem Grund werden wir in diesem Abschnitt die Kriterien immer in Abhängigkeit der Abstandsgrößen betrachten. Wichtig zu erwähnen ist, dass die Eingabe des Abstands absolut ist. Für die Visualisierung am Ende zählt aber natürlich der Relative Abstand im Verhältnis zu der Größe der Abbildung. Wenn es Absoluter Abstand von 10 Einheiten zwischen zwei Dateien ist, dann wirkt dieser bei einer Abbildung von 1000x1000 Einheiten natürlich deutlich kleiner als bei einer Abbildung von 200x200 Einheiten. Aus diesem Grund geben wir den Abstand immer Prozentual zur Größe an. Ein Wert von zum Beispiel einem Prozent bedeutet, dass der Abstand zwischen zwei Knoten 1% der Seitenlänge des Wurzelknotens beträgt. Dadurch, dass durch das Hinzufügen von Abständen, auch die Fläche des Wurzelknotens größer wird ist dieser Zusammenhang nicht linear, wie in Diagramm 1.14 zu sehen ist.

In Abbildung 1.15 und 1.16 zeigt, dass bei verschiedenen Softwares verschiedene Eingabe Abstände genutzt werden müssen, um den gleichen relativen Abstand zu erreichen.

Je nach Software Größe, Struktur und Ziel der Visualisierung kann es also sinnvoll sein, verschiedene Abstände zu wählen. Bei kleinen Softwareprojekten zum Beispiel kann es sinnvoll sein, auch größere relative Abstände zu

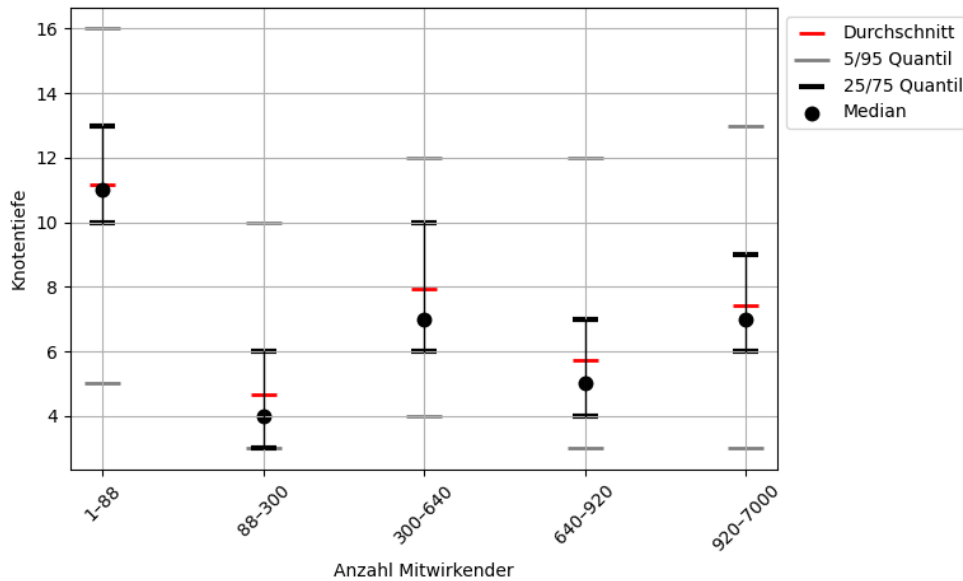


Abbildung 1.7: Die Anzahl der Mitwirkenden zeigt keinen erkennbaren Einfluss auf die Hierarchietiefe.

wählen, um die Struktur der Software besser erkennen zu können, weil man auch bei größeren Abständen die einzelnen Dateien und deren Werte gut erkennen kann (siehe Abbildung 1.17).

In dieser Arbeit werden wir daher die Kriterien immer in Abhängigkeit des Relativen Abstands betrachten, dabei werden wir Werte bis ca. 3% betrachten, da darüber hinaus die Abstände unnötig groß werden (siehe Abbildung 1.18).

Alles in allem Empfehlen wir also einen Abstand von 0.5% bis 3%, je nach Software und Ziel der Visualisierung.

1.5 Angestrebtes Seitenverhältnis

Wie bereits in den Grundlagen erwähnt (siehe Abschnitt ??) muss der angestrebte Seitenverhältniswert nicht zwingend 1 sein. Die bekannte d3.js Implementierung des Squarify Algorithmus [D3t] nutzt zum Beispiel den Goldenen Schnitt (1.618) als Zielwert für das Seitenverhältnis. Dieser Zielwert wird als Faktor in die Bedingung des Algorithmus eingebaut, der bestimmt, wann eine Reihe komplett ist, dabei ist dann das optimale Seitenverhältnis nicht 1, sondern eben der angestrebte Wert. Will wollen uns hier nicht streiten, ob 1 oder der Goldene Schnitt am Ende schöner und übersichtlichere Layouts erzeugt, sondern wir zweifeln an, dass diese Zielwerte überhaupt sinnvoll sind, da man wie am Ende von Abschnitt ?? an den Beispielen sieht, die Ziel Ratios nicht erreicht werden können (Ziel Seitenverhältnis von 1 führt zu einem durchschnittlichen Seitenverhältnis von 1,42 und Ziel Seitenverhältnis von 5 führt zu einem durchschnittlichen Seitenverhältnis von 2,79 - siehe Abbildung ?? und ??) speziell wenn wir uns Hierarchien anschauen, die tief verschachtelt sind und pro Ordner nur wenige Kindknoten

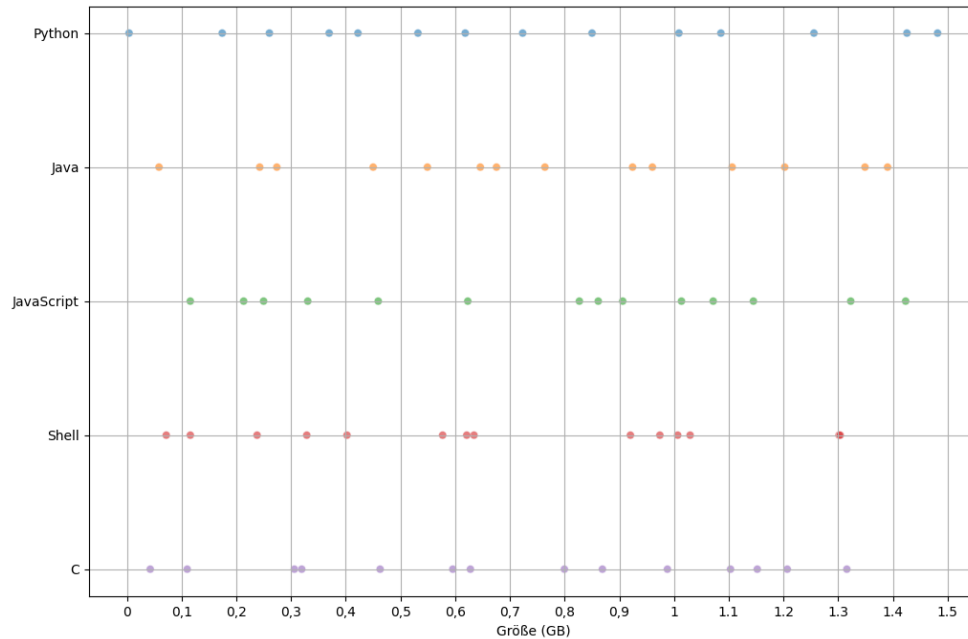


Abbildung 1.8: Größenverteilung der ausgewählten Repositories je Sprache mit gleichmäßiger Abdeckung bis 1,5 GB.

haben (siehe Datenanalyse Abschnitt 1.3), vermuten wir, dass diese Ziele un-
nereichbar sind, da die Struktur der Daten das Layout zu stark beeinflusst.

In Abbildung 1.19 ist der Zusammenhang zwischen dem angestrebten Seitenverhältnis und dem tatsächlich erreichten Seitenverhältnis abgebildet. Es ist klar zu erkennen, dass nie der angestrebte Wert erreicht wird, das tatsächliche Seitenverhältnis ist immer deutlich höher. Die Werte um 1 herum liefern dabei noch die besten Ergebnisse, trotzdem kommt hier kein Zielwert überhaupt in die Nähe von dem Goldenenschnitt geschweige denn 1. Ein Zielwert vom goldenen Schnitt liefert ein median Seitenverhältnis von 7.35 mit durchschnitt 18.80, was deutlich schlechter ist als ein Zielwert von 1 mit median 6.58 und durchschnitt 16.44. Interessant ist es, wenn man sich die Werte um 1 herum in Tabelle 1.2 anschaut.

Anders als erwartet liefert ein Angestrebtes Seitenverhältnis von 1 nicht das beste Ergebnis, sondern ein Wert von 1.05 liefert das beste Ergebnis. Es ist zwar im Median nur um ca. 5% besser, aber immerhin. Warum das der Fall ist, ist schwer nachzuvollziehen, vielleicht kann es in manchen Fällen besser sein leicht schlechtere Seitenverhältnisse auf höheren Ebenen zu haben, weil das dann zu besseren Seitenverhältnissen auf niedrigeren Ebenen führt, aber das ist nur eine Vermutung. Wir werden in dieser Arbeit den Wert von 1.05 als angestrebtes Seitenverhältnis verwenden, da dieser Wert die besten Ergebnisse für unsere Softwaremetrik liefert.

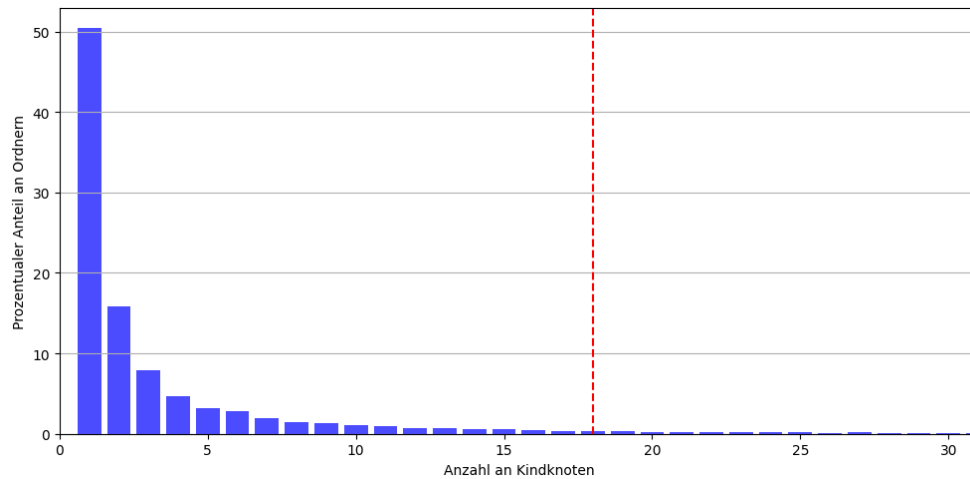


Abbildung 1.9: Kinderzahlen pro Ordner. Über 50% der Ordner haben nur ein Kind. Das 95%-Quantil liegt bei 18.

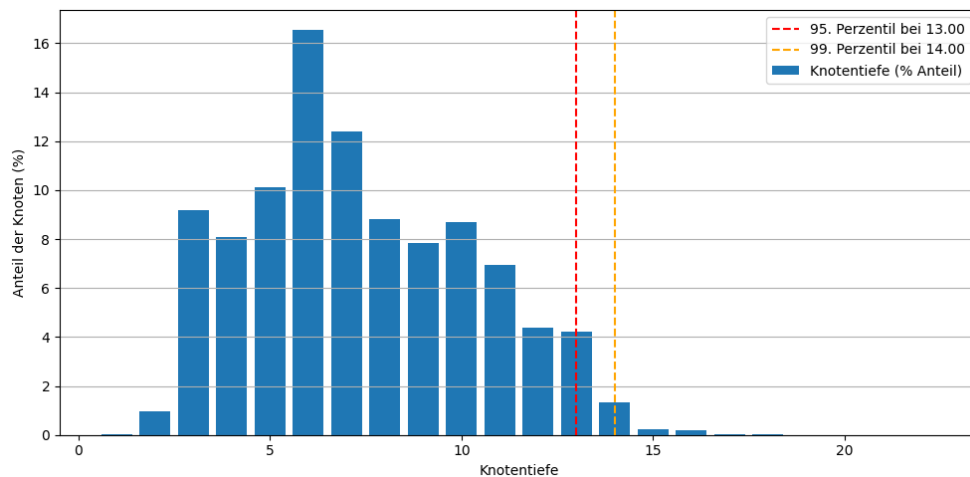


Abbildung 1.10: Verteilung der Tiefe. Die meisten Knoten liegen in mittleren Tiefen von 3 bis 11. Sehr tiefe Knoten sind selten.

1.6 Reihenfolge der Knoten im ersten Layoutschritt

Die Reihenfolge in der die Knoten in das Layout eingefügt werden, hat einen großen Einfluss auf das Layout. [JS91] haben in ihrem Paper herausgefunden, dass die ergebnisse am besten sind, wenn die Knoten in der Reihenfolge der Größe eingefügt werden. Also der Größte zuerst. Sie begründen das damit, dass sich dadurch bessere Seitenverhältnisse ergeben. Außerdem haben wir in Abschnitt 1.3 gezeigt, dass 95% der Ordner maximal 18 Kindknoten haben. Das sortieren aller Knoten in einem Ordner nach Größe ist also gar nicht mal so Zeitaufwendig, wie man denken könnte, wenn man tausende von Knoten hat, weil immer nur ein kleines Set von Knoten sortiert werden muss. Kleines Beispiel: Wenn wir von einem typischen Sortieralgorithmus mit einer Laufzeit von $\mathcal{O}(n \log n)$ (zB. quicksort), einer Anzahl von 4221 Kno-

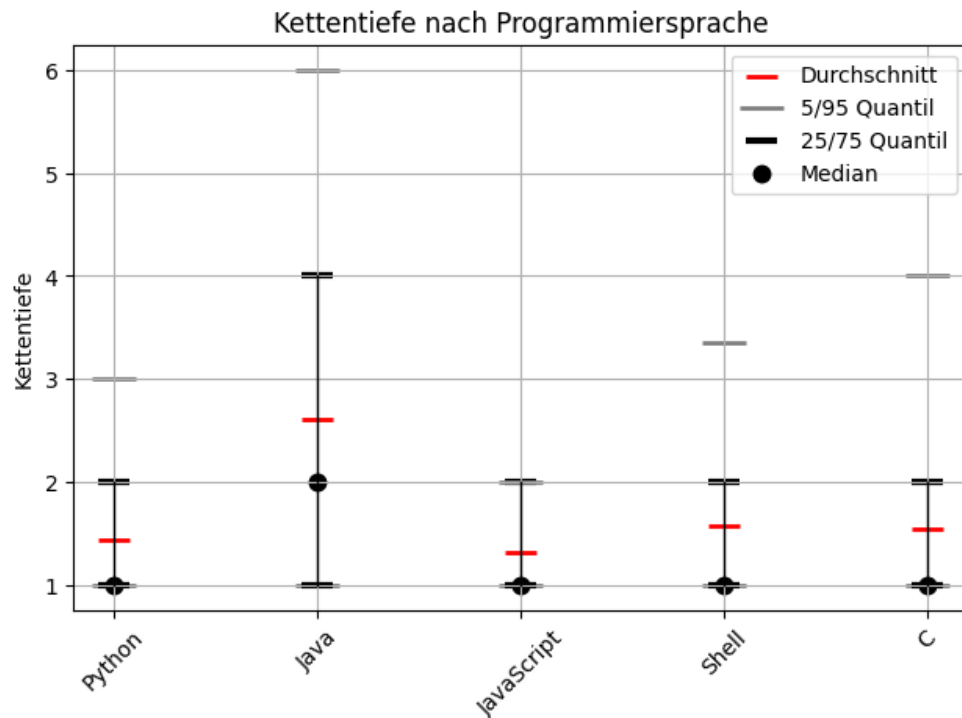


Abbildung 1.11: Kettentiefe nach Sprache. Java zeigt höhere Mittelwerte und größere Varianz.

ten (Durchschnittswert der Anzahl der Knoten siehe Abschnitt 1.3) und 18 Knoten pro Ordner. Wenn wir alle Knoten auf einmal sortieren wollen:

$$T_1 = n \cdot \log_2(n) = 4221 \cdot \log_2(4221)$$

$$\log_2(4221) \approx 12.04$$

$$T_1 \approx 4221 \cdot 12.04 \approx 50821$$

Wenn wir die Knoten in 18er bundleln sortieren wollen:

$$T_2 = m \cdot (k \cdot \log_2(k)) = 235 \cdot (18 \cdot \log_2(18))$$

$$\log_2(18) \approx 4.17$$

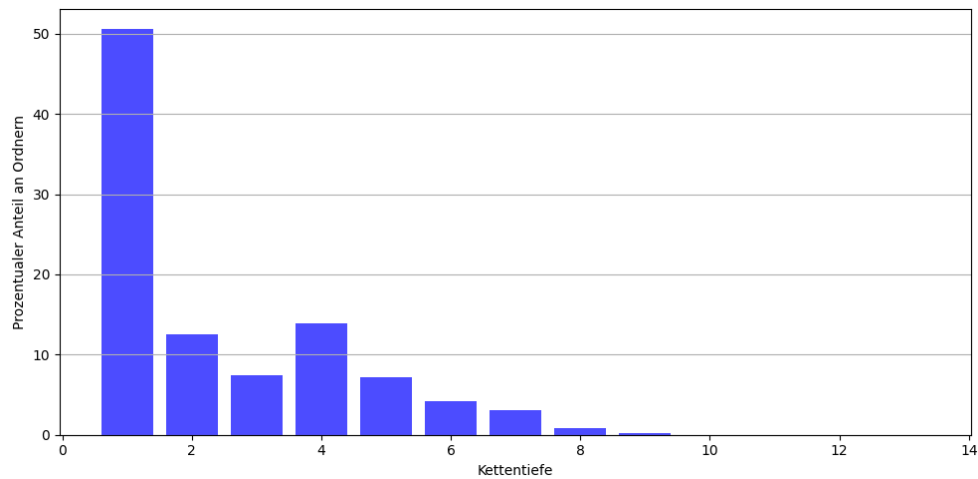


Abbildung 1.12: Verteilung der Kettenlängen. Viele Ketten sind kurz. Ketten der Länge 4 treten ebenfalls häufig auf.

Angestrebtes Seitenverhältnis	Median des Seitenverhältnisses	Durchschnittliches Seitenverhältnis
0.900	6.82	16.32
0.990	6.57	16.44
1.000	6.58	16.44
1.025	6.37	16.55
1.050	6.28	16.17
1.100	6.30	16.17
1.150	6.32	16.67
1.200	6.40	16.78
1.300	6.79	16.89

Tabelle 1.2: Durchschnittliches und median Seitenverhältnis bei verschiedenen angestrebten Seitenverhältnissen um 1 herum bei der Anwendung des Standard Squarify Algorithmus auf die ausgewählten Repositories.

$$T_2 \approx 235 \cdot (18 \cdot 4.17) = 235 \cdot 75.06 \approx 17639$$

D3.js hat das nicht im standard algorithmus implementiert, aber in der dokumentation des Squarify Algorithmus steht explizit:

You probably also want to call `root.sort` to order the hierarchy before computing the layout. [\[D3t\]](#)

Aus diesem Grund testen wir jetzt, ob das sortieren der Knoten nach Größe wirklich die besten auch für unsere Daten liefert und ob sich das auch auf andere Kriterien als das Seitenverhältnis auswirkt.

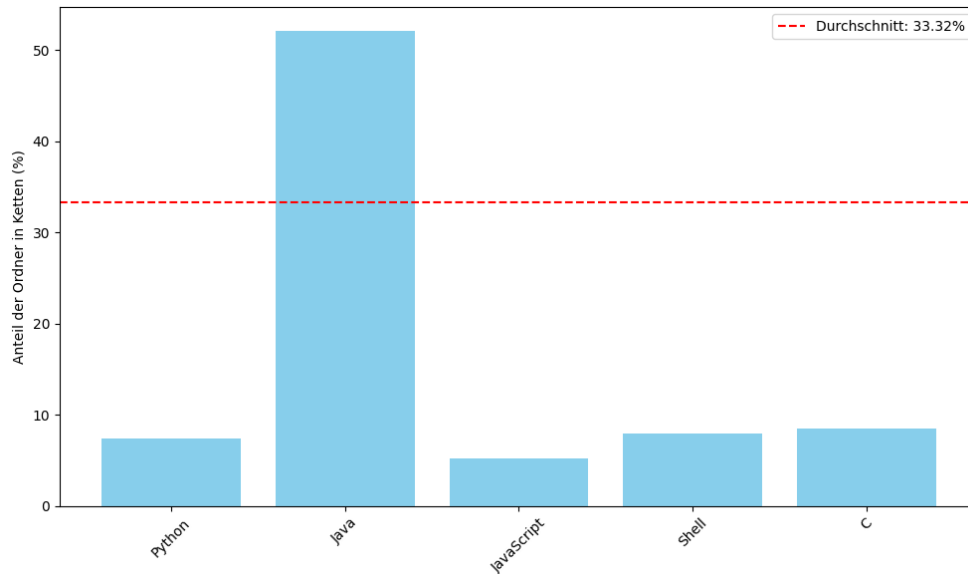


Abbildung 1.13: Anteil der Ordner, die Teil einer Kette sind. Java liegt mit etwa 50% deutlich über dem Mittel von etwa 33%.

1.7 Zweifache Berechnung

Die Grundlegende Idee dieser Erweiterung ist es, dass sich die Fläche der Knoten mit Abstand durch das Layout und die Fläche der Knoten ohne Abstand approximieren lässt. Die Idee ist es also einen ersten Durchlauf zu machen, bei dem das Layout ohne Abstand berechnet wird. Dann werden die Größen der Knoten entsprechend dem Layout angepasst, sodass die Größe der Knoten nun auch den Abstand berücksichtigt. Anschließend wird ein zweiter Layout-Schritt mit diesen angepassten Größen durchgeführt, um das finale Layout zu berechnen.

Bevor wir uns die Details und Ergebnisse dieser Erweiterung anschauen, wollen wir vorweg nehmen, dass diese Erweiterung natürlich nicht optimal funktionieren kann und das auch klar ist, da sich die Änderung der Größe der Knoten natürlich auch das Layout im Zweiten Durchlauf ändern wird, wodurch die Größen der Knoten wieder nicht korrekt sind. Was ja überhaupt erst das Grundlegende Problem ist (siehe Abschnitt ??). Allerdings ist es ein erster Schritt sich dem Problem zu nähern und zu schauen, ob es sich lohnt in diese Richtung weiter zu forschen.

Der Grundlegende Algorithmus bleibt also (fast) gleich, nur dass zwischen dem ersten und dem zweiten Durchlauf ein zusätzlicher Schritt *Größenanpassung* eingefügt wird. Die Einzige Änderung die vorgenommen werden muss ist, dass Knoten nur mit dem definierten Abstand zwischen dem Elternknoten platziert werden können und generell die Fläche des Elternknotens um den Abstand verkleinert wird. Wir bezeichnen den Schritt bei dem der Squarify Algorithmus durchgeführt wird als *Layout-Schritt* und den Schritt bei dem die Größen der Knoten angepasst werden als *Größenanpassung*. Außerdem ist es nötig nach dem zweiten Durchlauf die Knoten, deren

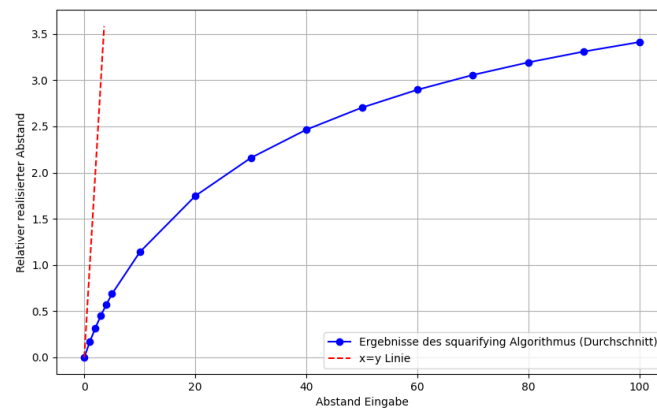


Abbildung 1.14: Zusammenhang zwischen der Absoluten Abstandeingabe und dem Relativen Abstand im Verhältnis zur Seitenlänge des Wurzelknotens

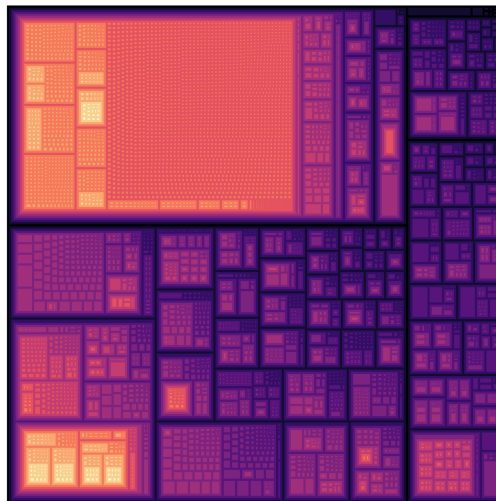


Abbildung 1.15: Layout mit einem Absoluten Abstand von 200 und einem Relativen Abstand von 0,4 (generiert aus dem React Repository [[Gita](#)])

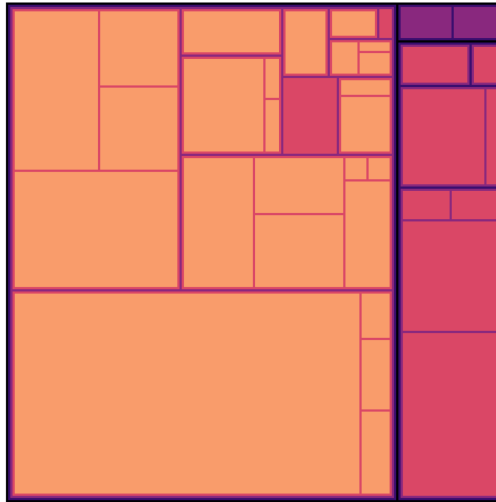


Abbildung 1.16: Layout mit einem Absoluten Abstand von 4 und einem Relativen Abstand von 4 (generiert aus dem Qwen3 Repository [[Gitb](#)])

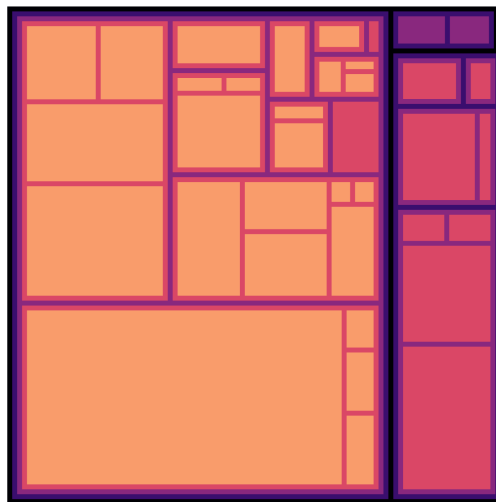


Abbildung 1.17: Layout mit einem Absoluten Abstand von 10 und einem Relativen Abstand von 1 (generiert aus dem Qwen3 Repository [[Gitb](#)])

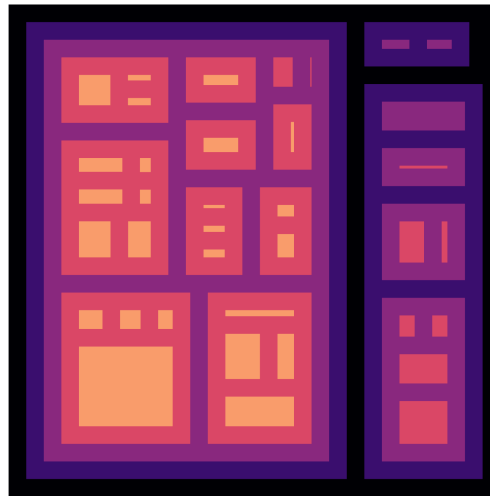


Abbildung 1.18: Layout mit einem Absoluten Abstand von 100 und einem Relativen Abstand von 35 (generiert aus dem Qwen3 Repository [[Gitb](#)])

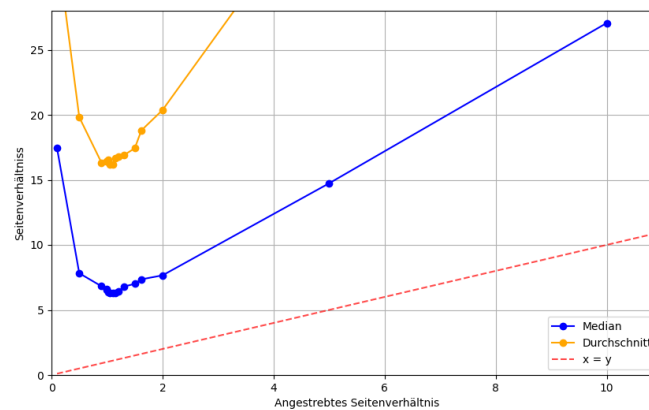


Abbildung 1.19: Zusammenhang zwischen dem angestrebten Seitenverhältnis und dem tatsächlich erreichten Seitenverhältnis bei der Anwendung des Standard d3.js Squarify Algorithmus auf die ausgewählten Repositories.

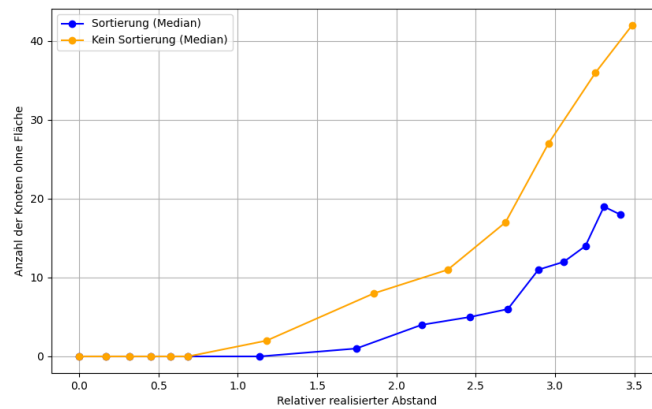


Abbildung 1.20: Anzahl der fehlenden Knoten in Abhängigkeit des relativen Abstands für Sortierung nach Größe und keiner Sortierung.

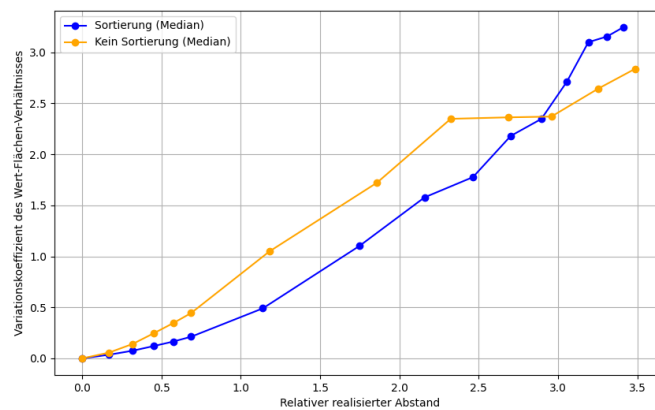


Abbildung 1.21: Varianzkoeffizient des Flächenverhältnisses in Abhängigkeit des relativen Abstands für Sortierung nach Größe und keiner Sortierung.

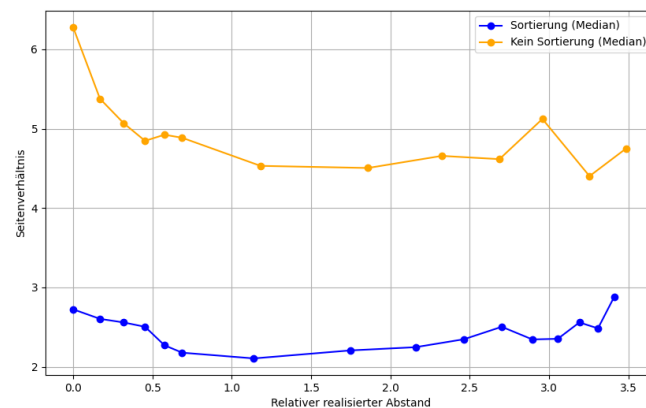


Abbildung 1.22: Durchschnittliches Seitenverhältnis in Abhängigkeit des relativen Abstands für Sortierung nach Größe und keiner Sortierung.

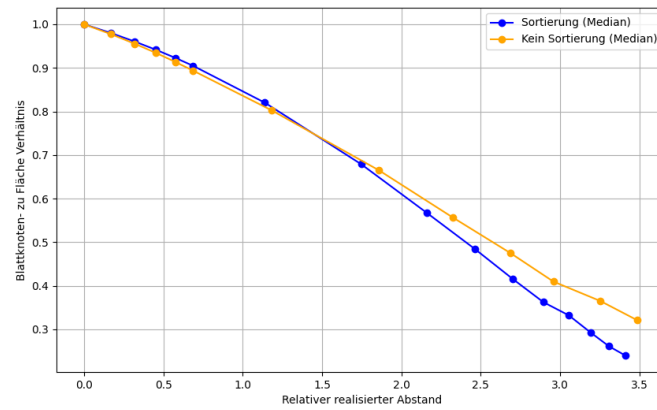


Abbildung 1.23: Platznutzung in Abhängigkeit des relativen Abstands für Sortierung nach Größe und keiner Sortierung. Werte näher bei 1 sind besser.

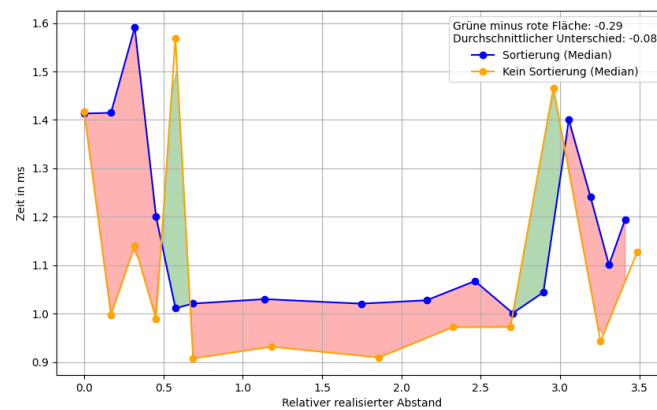


Abbildung 1.24: Zeitaufwand in Abhängigkeit des relativen Abstands für Sortierung nach Größe und keiner Sortierung.

größenwert ja nun den abstand beinhalten, zu verkleinern, um auch den abstand zwischen den Geschwistern herzustellen. Es ist zu erkennen, dass dadurch der Abstand sowohl zwischen Geschwistern als auch zu den Elternknoten den doppelten wert des definierten Abstands hat, dieses Problem ignorieren wir hier, da man es trivialerweise lösen könnte, indem man immer nur die hälfte des Abstands zwischen Geschwistern und Elternknoten abzieht, was wir hier der einfachheit halber nicht tun. – ODER VIELLEICHT HIER IN DER THESIS DOCH? DANN KÖNNTE ICH MIR DIESEN ABSCHNITT SPAREN, AUCH WENN ES IN DER IMPLEMENTIERUNG AM ENDE ANDERS IST

Wir stellen verschiedene Ansätze vor, was sowohl die Größenanpassung als auch die Anpassung der Knoten nach dem zweiten Durchlauf angeht. Die Algorithmen funktionieren allerdings alle nach ähnlichem Prinzip: Es wird zunächst für jeden Knoten die Fläche die der Abstand in diesem Layout benötigen würde addiert, indem die Fläche aus der neuen Länge (alte Länge + 2 mal den Abstand) und der neuen Breite (alte Breite + 2 mal den Abstand) berechnet wird. Zusätzlich wird für Elternknoten die Flächenvergrößerung aller Kinderknoten addiert. An dieser Stelle ist allerdings nicht klar, wie sich die Flächenvergrößerung der Kinderknoten auf die Fläche der Elternknoten auswirkt, da diese Änderung selbst von der Anordnung der Kinderknoten abhängt. Wir testen verschiedene Ansätze, um die Flächenänderung der Elternknoten in Abhängigkeit zu der Flächenänderung der Kinderknoten zu approximieren.

Nach dem zweiten Durchlauf wird nun die Fläche der Knoten so reduziert, dass sowohl der Abstand zwischen Geschwistern als auch der Abstand zu den Elternknoten den gewünschten Wert hat. Dies ist straight forward und wird hier nicht weiter erläutert. Anzumerken ist aber das dieser Schritt speziell abhängt von der Art der Größenanpassung, die im ersten Schritt durchgeführt wurde.

1.7.1 Größenanpassung

Es gibt verschiedene Ansätze, wie die Größenanpassung durchgeführt werden kann. Grundsätzliches Ziel der Größenanpassung ist es, die Fläche der Knoten so zu vergrößern, dass diese Fläche den Abstand zwischen den Knoten möglichst gut approximiert. Wir testen hier zwei Ideen aus.

Der erste ansatz ist ähnlich zu dem cascaded ansatz. Die Fläche der Knoten wird um den Abstand in beiden Richtungen vergrößert. Zusätzlich wird die Fläche der Elternknoten um die Fläche der Kinderknoten vergrößert. wir zeigen genau auf, warum dieser Ansatz große Schwächen hat. bzw Pseudocode für den Algorithmus [1](#).

Algorithm 1 Absolute Größenanpassung

```

1: function ABSOLUTEVALUEINCREASE(node: SquarifyNode, margin: num-
   ber)
2:   childrenValueIncrease  $\leftarrow$  0
3:   if node.children then
4:     for child in node.children do
5:       childrenValueIncrease += absoluteValueIncrease(child, mar-
   gin)
6:     end for
7:   end if
8:   valueIncrease  $\leftarrow$  width * margin * 2 + length * margin * 2 + margin *
   margin * 4 + childrenValueIncrease
9:   node.value += valueIncrease
10:  return valueIncrease
11: end function

```

Das Problem des Algorithmuses ist am besten an einem Beispiel zu verdeutlichen. Wir visualisieren den ZWITEN AnHANG - auch wieder eine händisch erstellte Map, um das Problem zu verdeutlichen. In Abbildung 1.25 ist das Layout mit einem Abstand von 0 zu sehen.



Abbildung 1.25: Treemap Layout generiert mit dem Squarify Algorithmus nach Abschnitt ?? mit einem Abstand von 0 und der einfachen Größenanpassung auf der händisch erstellten map (siehe Anhang).

In Abbildung 1.26 ist das Layout mit einem Abstand von 1 zu sehen. Es ist zu erkennen, dass die Knoten auf der linken Seite des Layouts sich außerhalb ihrer Elternknoten erstrecken. Warum passiert das? Knoten 10 ist im zweiten Layout-Schritt deutlich schmaler als im ersten Layout-Schritt. Da-

durch wird die Fläche die durch den Abstand eingenommen wird größer als angenommen, weshalb die Fläche des Knotens nach abzug des Abstands im letzten Schritt kleiner ist, als gewünscht. Dementsprechend ist auch die Fläche der Knoten unten links größer als gewünscht, da das Layout der Knoten im zweiten Layout-Schritt quadratischer wird. Knoten 5, der Knoten, der am Quadratischsten ist, wird also am größten erscheinen. Obwohl beide den selben wert haben ist Knoten 5 ca. 1,2 mal größer als Knoten 10) In diesem Beispiel erscheint der unterschied kaum merklich, aber es gibt ihn trotzdem und in anderen fällen kann dieser Unterschied merklich werden. Viel signifikanter ist aber der Effekt, dass Elternknoten ebenfalls immer schmaler werden, wodurch die fläche, die der innerere abstand einnimmt, ebenfalls größer wird und dass sogar immer mehr von ebene zu ebene, wenn man runter geht. Dadurch wird die Fläche für die Kindknoten immer kleiner, was dazu führt, dass Knoten teilweise über ihre Elternknoten hinauswachsen.

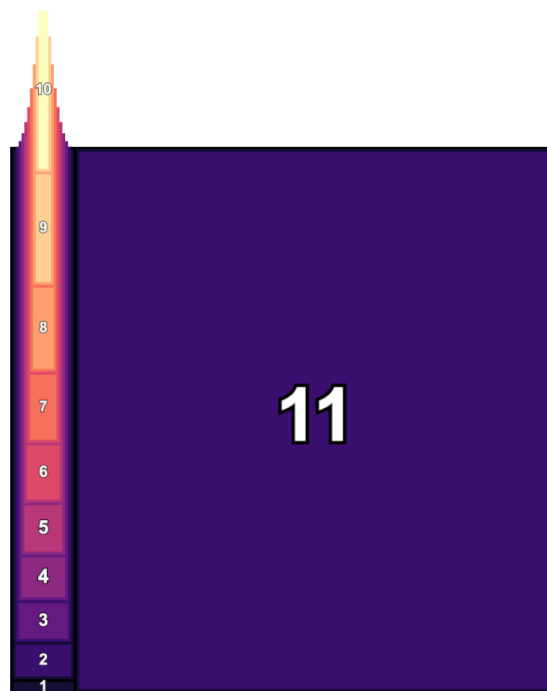


Abbildung 1.26: Treemap Layout generiert mit dem Squarify Algorithmus nach Abschnitt ?? mit einem Abstand von 1 und der absoluten Größenanpassung auf der händisch erstellen map (siehe Anhang).

Dieser Effekt kann einfach behoben werden, wie wir in Abschnitt 1.7.1 sehen werden.

Bei der berechnung zuvor wurde einfach der margin fläche eines Elternknotens berechnet, auf basis der alten Fläche, ohne die Flächenänderung durch die Kinderknoten zu berücksichtigen. dadurch wird die resultierende Fläche der Elternknoten zu klein sein, da die Fläche der Kinderknoten nicht berücksichtigt wird. Wir stellen folgend eine zweite Version der Größenanpassung vor, die versucht die Fläche der Elternknoten durch die Fläche der Kinderknoten zu approximieren. Wir werden dann beide Versionen verglei-

chen und schauen, welche besser funktioniert. Die Idee bei dieser Erweiterung ist es die Änderung der Kinder schon vor dem Hinzufügen der Abstandsfläche zu berücksichtigen. Dafür muss die relative Flächenänderung durch die Kindknoten berechnet werden, und damit dann die Seitenlängen anpassen und dann die margins hinzufügen, um die neue Fläche zu erhalten. Der Algorithmus wird im foldenden als Pseudocode dargestellt (siehe Algorithmus 2).

Algorithm 2 Relative Größenanpassung

```

1: function RELATIVEVALUEINCREASE(node: SquarifyNode, margin: num-
   width
ber)
2:   width  $\leftarrow$  node.x1 - node.x0
3:   length  $\leftarrow$  node.y1 - node.y0
4:   if node.isLeaf then
5:     valueIncrease  $\leftarrow$  width * margin + length * margin + margin *
margin * 2
6:     node.value += valueIncrease
7:     return valueIncrease
8:   end if
9:   childrenValueIncrease  $\leftarrow$  0
10:  if node.children then
11:    for child in node.children do
12:      childrenValueIncrease += relativeValueIncrease(child, margin)
13:    end for
14:  end if
15:  ratioChildrenValueIncrease  $\leftarrow$  (node.value + childrenValueIncrease)
/ node.value
16:  valueIncrease  $\leftarrow$  Math.sqrt(ratioChildrenValueIncrease) * width *
margin * 2 + Math.sqrt(ratioChildrenValueIncrease) * length * margin
* 2 + margin * margin * 4 + childrenValueIncrease
17:  node.value += valueIncrease
18:  return valueIncrease
19: end function

```

Problem: Der Algorithmus in dieser Form zeigt einige Probleme auf. Die Flächenvergrößerung der Kindknoten sagt nichts darüber aus, in welche Richtung sich die Fläche ändert. Es wird davon ausgegangen, dass sich die Fläche gleichmäßig in beide Richtungen ändert (siehe Zeile 13 und 14 in Algorithmus ZEILEN ANPASSEN 2). es kommt also zu ähnlichen Problem wie davor. es können knoten sowohl zu viel platz als auch zu wenig Platz bekommen, jenachdem ob Sie im zweiten schritt quadratischer oder schmaler werden. Siehe Abbildung 1.27 für ein Beispiel.

Scaling der Knoten

Beide Ansätze der Größenanpassung haben das Problem, dass die Fläche der Knoten nicht exakt die Fläche ist, die der Abstand benötigt. Dadurch können Probleme entstehen.

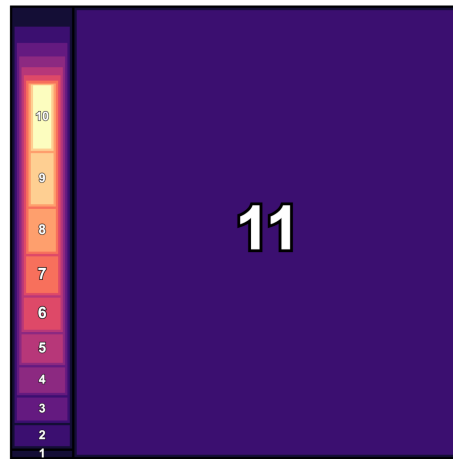


Abbildung 1.27: Treemap Layout generiert mit dem Squarify Algorithmus nach Abschnitt ?? mit einem Abstand von 1 und der relativen Größenanpassung auf der händisch erstellen map (siehe Anhang).

Wenn die Fläche innerhalb von Elternknoten immer kleiner wird, kann es passieren, dass Knoten über ihre Elternknoten hinauswachsen, wie in Abbildung 1.26 zu sehen ist. Es kann genauso passieren, dass die Fläche innerhalb der Knoten größer wird, wie in Abbildung 1.27 auf der linken Seite zu erkennen ist. Dieser Effekt kann trivialerweise behoben werden, indem der zweite Layout-Schritt angepasst wird, sodass die Knoten immer auf die Fläche des Elternknotens skaliert werden. Vor jeden Squarify-Schritt wird dafür die wirklich zur Verfügung stehende Fläche des Elternknotens berechnet und die Kindknoten entsprechend dieser Änderung skaliert, sodass sie genau in die Fläche des Elternknotens passen.

Der Nachteil dieser Methode ist in Abbildung 1.28 zu erkennen. Die Knoten werden dadurch natürlich nicht mehr proportional zu ihren Werten sein. Knoten 10 hat zum Beispiel ein Verhältnis von ca. 0.5 zu seinem Wert, während Knoten 6 ein Verhältnis von ca. 1.4 zu seinem Wert hat.

Je genauer die Größenanpassung der Knoten ist, desto geringer fällt natürlich dieser Effekt aus. Siehe im Vergleich dazu Abbildung 1.29, da die relative Größenanpassung deutlich genauer ist, ist der Effekt hier auch deutlich geringer. Knoten 10 hat Größe 40 und Knoten 6 hat Größe 41, was fast ähnlich groß ist.

Persönlich würde ich sagen, dass dieser Effekt für das Herunter skalieren der Knoten gut ist, da sonst eine grundlegende Eigenschaft der Darstellung verletzt wird, aber für das Hoch skalieren der Knoten könnte man auch sagen, dass es wichtiger ist, die Fläche der Knoten proportional zu ihren Werten zu halten, als die *verschwendete* Fläche aufzufüllen. Wir entscheiden uns in dieser Arbeit dafür, dass wir die Knoten immer skalieren, auch nach oben, um die Identität der Darstellung von Treemap-Layouts zu wahren. Dadurch tauschen wir sozusagen Proportionalität der Knoten gegen Platznutzung ein.

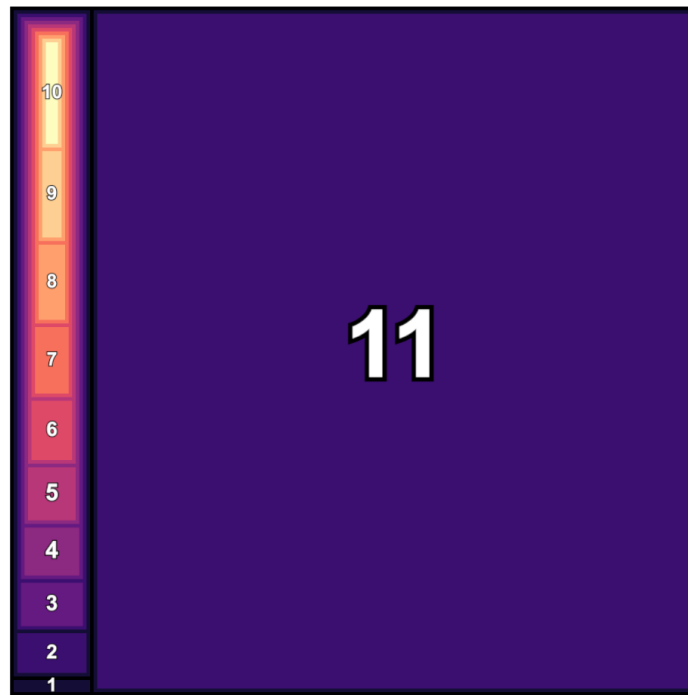


Abbildung 1.28: Treemap Layout generiert mit dem Squarify Algorithmus nach Abschnitt ?? mit einem Abstand von 1 und der einfachen Größenanpassung auf der händisch erstellen map (siehe Anhang) und der Skalierung der Knoten.

Vergleich der Ansätze

Wir vergleichen diese beiden Ansätze jetzt mit dem Standard Squarify nested squarify algorithmus von d3.js, wie er von Mike Bostock implementiert wurde [Bos23].

In Abbildung 1.30 klar zu erkennen, dass das beide Erweiterungen des Squarify Algorithmus deutlich weniger fehlende Knoten haben als der Standard Squarify Algorithmus von d3.js. Eine wirklich extreme Verbesserung ist zu erkennen, die speziell bei größeren Abständen noch stärker wird. Während der Standard Squarify Algorithmus bei einem Abstand von 1% im Schnitt ca. 600 fehlende Knoten hat, haben die beiden Erweiterungen fast keine fehlenden Knoten mehr. Zudem ist zu erkennen, dass die absolute Größenanpassung bis 2,5% etwas mehr fehlende Knoten hat als die relative Größenanpassung. Außerdem ist zu erkennen, dass die relative Größenanpassung auch noch bei größeren Abständen weniger fehlende Knoten hat und konstanter bleibt, während die absolute Größenanpassung bei Abständen über 2,5% höhere Ergebnisse erzielt.

Bei allen drei Algorithmen ist zu erkennen, dass der Varianzkoeffizient des Flächenverhältnisses mit zunehmendem Abstand steigt (siehe Abbildung 1.31). Das bedeutet, dass die Fläche der Knoten mit zunehmendem Abstand immer unproportionaler zu ihren Werten wird. Klar zu erkennen ist, dass das schlechteste Seitenverhältnis durch den d3.js squarify Algorithmus

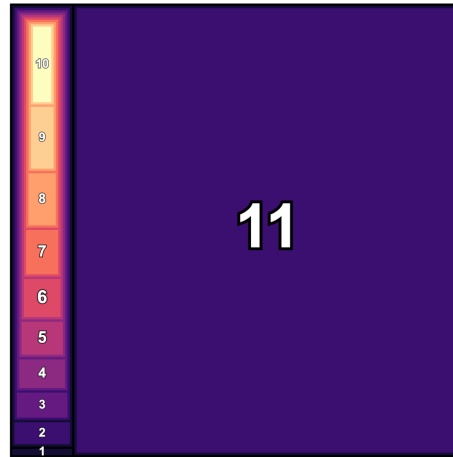


Abbildung 1.29: Treemap Layout generiert mit dem Squarify Algorithmus nach Abschnitt ?? mit einem Abstand von 1 und der relativen Größenanpassung auf der händisch erstellten map (siehe Anhang) und der Skalierung der Knoten.

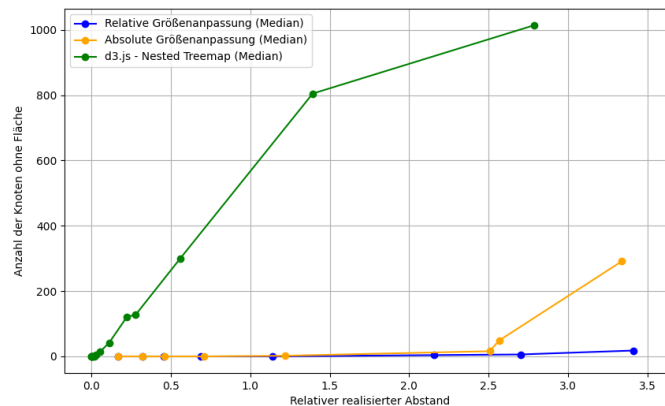


Abbildung 1.30: Anzahl der fehlenden Knoten in Abhängigkeit des relativen Abstands für die drei verschiedenen Größenanpassungen.

mus erzielt wird. Zweit bestes durch die relative Größenanpassung und das beste durch die absolute Größenanpassung (siehe Abbildung 1.31).

In Abbildung 1.32 ist zu erkennen, dass der d3.js ab ca. 0,5% Abstand Seitenverhältnisse von 4,5 bis 5 erzielt, während die beiden Erweiterungen des Squarify

In Abbildung 1.33 ist zu erkennen, dass die Platznutzung bei den beiden Erweiterungen dauerhaft näher an der 1 ist. Theoretisch hätten wir erwartet, dass alle drei Varianten ähnliche Werte produzieren, da der relative Abstand ja bei allen gleich ist. Wir erklären uns das dadurch, dass ja auch mehr Knoten fehlen, wodurch dann die Fläche der Knoten auch kleiner wird.

In Abbildung 1.34 ist zu erkennen, dass der Zeitaufwand bei den beiden Erweiterungen dauerhaft höher ist. Teilweise doppelt so hoch, teilweise weniger. Das ist natürlich zu erwarten, da ja ein zweiter Layout-Schritt durch-

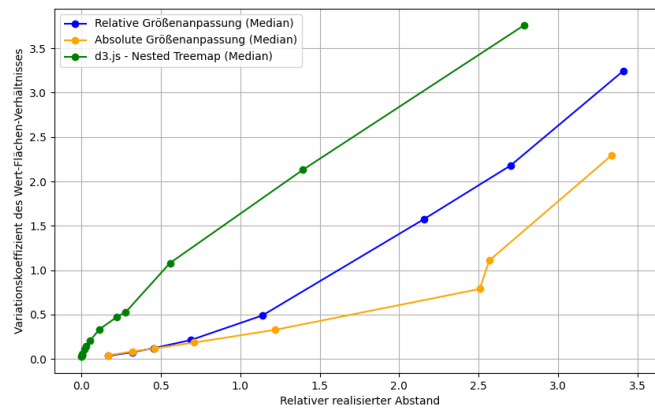


Abbildung 1.31: Variationskoeffizient des Flächenverhältnisses in Abhängigkeit des relativen Abstands für die drei verschiedenen Größenanpassungen.

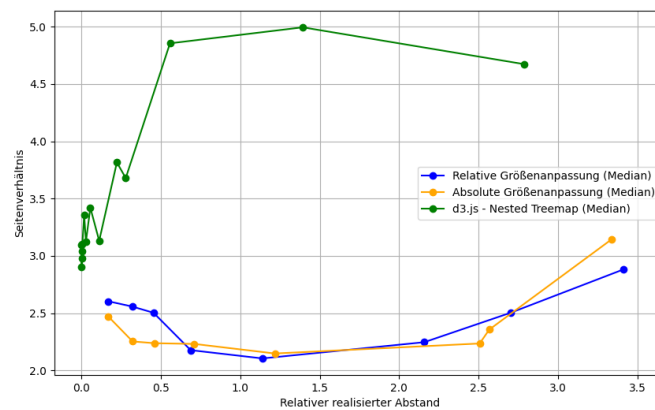


Abbildung 1.32: Durchschnittliches Seitenverhältnis in Abhängigkeit des relativen Abstands für die drei verschiedenen Größenanpassungen.

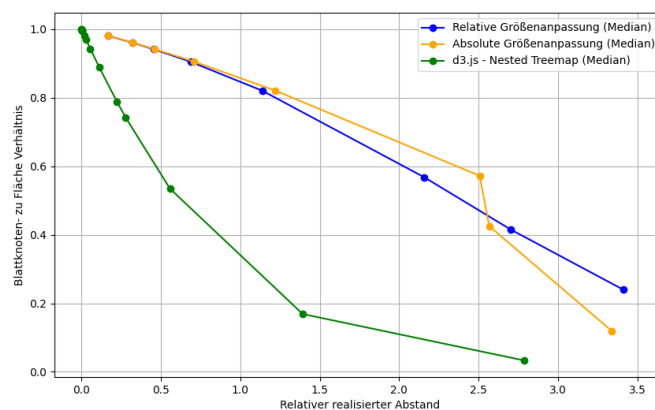


Abbildung 1.33: Platznutzung in Abhängigkeit des relativen Abstands für die drei verschiedenen Größenanpassungen. Werte näher bei 1 sind besser.

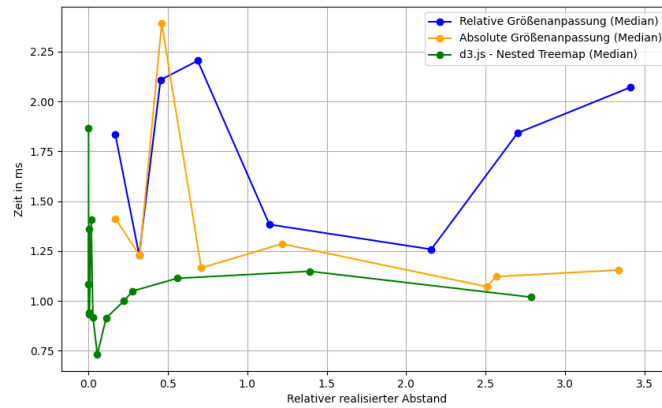


Abbildung 1.34: Zeitaufwand in Abhängigkeit des relativen Abstands für die drei verschiedenen Größenanpassungen.

geführt wird. Die relative Größenanpassung ist dabei etwas aufwändiger als die absolute Größenanpassung, da hier ja auch die relative Flächenänderung der Kindknoten berechnet werden muss. Allerdings ist der Unterschied hier nicht so groß. Anzumerken ist, dass das verschwinden von Knoten keinen Einfluss auf die Laufzeit des d3.js Squarify Algorithmus hat, da dieser zuerst das Layout berechnet und dann die Abstände abzieht, bei den angepassten Versionen ist. In den angepassten Versionen implementieren wir early stopping, sobald die Fläche eines Knotens 0 oder negativ wird, was aber nicht so oft passiert, dass es einen großen Einfluss auf die Laufzeit hat.

Alles in allem schneiden die beiden Erweiterungen des Squarify Algorithmus deutlich besser ab als der Standard Squarify Algorithmus von d3.js. Die absolute Größenanpassung schneidet dabei beim Wert-Flächen-Verhältnis besser ab, während die relative Größenanpassung bei der Anzahl der fehlenden Knoten besser abschneidet. Beide Erweiterungen sind valide und zu Begründende alternativen zum Standard Squarify Algorithmus von d3.js. Für die weitere Arbeit in dieser Thesis entscheiden wir uns für die relative Größenanpassung, da wir hier die Anzahl der fehlenden Knoten als das wichtigste Kriterium ansehen.

In den wichtigsten Kriterien, Anzahl der fehlenden Knoten (siehe Abbildung 1.20), der Varianzkoeffizient des Flächenverhältnisses (siehe Abbildung 1.21) und das durchschnittliche Seitenverhältnis (siehe Abbildung 1.22) schneidet die Sortierung nach Größe durchweg besser ab, weshalb wir uns dafür entscheiden, die Knoten immer nach Größe zu sortieren.

1.8 Platzierung der Knoten nach erstem Layoutschritt

Der Algorithmus ihrer Arbeit über Cascaded Treemap von Lu und Fogarty [LFo8], wird die Platzierung der Knoten im zweiten Layout-Schritt beibehalten. Im Grunde merken Sie sich in welcher Reihe an welcher Position jeder Knoten platziert ist und behalten diese Platzierung bei. D3.js stellt zu ihrem

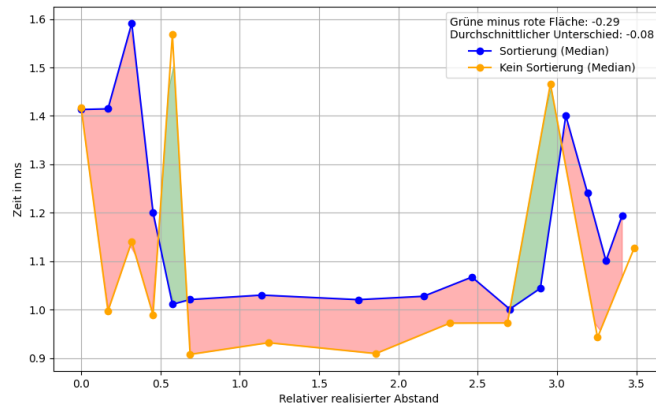


Abbildung 1.35: Zeitaufwand in Abhängigkeit des relativen Abstands für Sortierung nach Größe und keiner Sortierung.

Squarify Algorithmus auch einen Resquarify Algorithmus zu Verfügung, der im Grunde diesen zweiten Layout-Schritt mit beibehaltung der Platzierung der Knoten durchführt [D3t]. D3.js begründet die existenz dieser Strategie anders als Lu und Fogarty, nämlich damit, dass ein erneutes berechnen der Layouts mehr Zeit in anspruch nimmt. Dies macht Sinn weil wenn die Platzierung gleich bleibt, muss ich ja nur noch die Breite der Reihen und die Höhe der Knoten anpassen und nicht noch verschiedene Positionierungen testen und vergleichen.

Die Qualität dieser Strategie wird leider nicht separat evaluiert und hinterfragt. Wir stellen die Hypothese auf, dass durch diese Strategie aufgrund der verschiedenen Flächenänderungen schlechtere Seitenverhältnisse entstehen können, da die optimale Reihenfolge der Knoten sich ja durch die Flächenänderung ändern kann. Wir wollen also nachprüfen ob dieser Ansatz wirklich der beste ist, oder ob es nicht vielleicht bessere Strategien für den zweiten Layout-Schritt gibt.

Durch die updates der Größen der Knoten nach dem ersten Layoutschritt, kann es passieren, dass die Größenreihenfolge der Knoten sich ändert. Das als zum Beispiel Knoten A im ersten Layoutschritt größer ist als Knoten B, aber nach dem Update der Größen von Knoten A kleiner ist als Knoten B. Das kann dazu führen, dass im folgenden Layoutschritt, die Knoten in einer anderen Reihenfolge platziert werden als im ersten Layoutschritt. Das ist erstmal gut für die Seitenverhältnisse der Knoten (wenn wir davon ausgehen, dass die Reihenfolge nach größe sortiert optimla ist), allerdings kann es dazu führen, dass Knoten an ganz anderen Positionen platziert werden, als im ersten layoutschrift. Das kann dazu führen, dass sich die Form ändert und das kann dazu führen, dass die Kinder anders platziert werden etc. wodurch sich die benötigten Abstände ändern, wodurch die vorausberechnete Fläche der Knoten nicht mehr passt und Knoten entweder verschwinden oder die Wert Fläche relation schlechter wird. Das kann natürlich genauso passieren, wenn die Knoten in der selben Reihenfolge platziert werden, wie im ersten

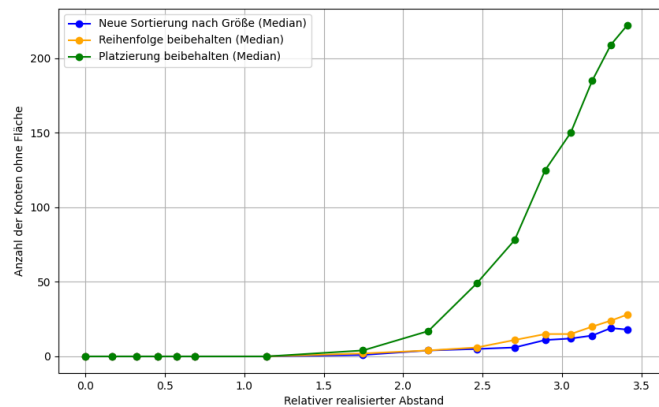


Abbildung 1.36: Anzahl der fehlenden Knoten in Abhängigkeit des relativen Abstands für die drei verschiedenen Platzierungsstrategien.

Layoutschritt. Es ist nicht klar erstichtlich, was hier am Ende des Tages am besten ist. Deswegen implementieren wir 3 verschiedene Ansätze:

- **Reihenfolge der Knoten nach Größe:** Die Knoten werden in der Reihenfolge ihrer neu berechneten Größe platziert. Wir vermuten, dass dadurch die besten Seitenverhältnisse entstehen, aber die schlechtesten Wert-Fläche-Relationen, weil sich die Seitenverhältnisse im Vergleich zum ersten Layoutschritt stark ändern können und dadurch die vorausberechnete Fläche der Knoten nicht mehr passt.
- **Reihenfolge der Knoten beibehalten:** Die Knoten werden in der Reihenfolge platziert, in der sie im ersten Layoutschritt platziert wurden (absteigend sortiert). Wir vermuten, dass dadurch im Großen und Ganzen die Platzierung der Knoten gleich bleibt, außer es gibt wirklich große Flächen updates. Wir vermuten, dass dies ein guter mittelweg zwischen dem ersten und dem jetzt folgenden ansatz ist.
- **Platzierung der Knoten beibehalten:** Nach dem ersten Layoutschritt wird sich genau die Aufteilung der Knoten in Reihen und die Platzierung der Knoten in den Reihen gemerkt. Wir vermuten, dass dadurch die Form der Knoten im großen und ganzen ähnlich zum ersten Schritt bleibt, wodurch die Wert-Fläche-Relationen besser bleibt, aber die Seitenverhältnisse schlechter werden könnten, die die optimale Reihenfolge der Knoten (aufgrund der neuen werte) nicht mehr berücksichtigt wird.

•

Es ist wie gesagt nicht ganz klar, was genau die Auswirkungen sind. Deswegen nur Vermutungen, die wir jetzt testen werden.

In Abbildung 1.36 ist klar zu erkennen, dass die Strategie, die Knoten in der Platzierung beizubehalten, die meisten fehlenden Knoten produziert.

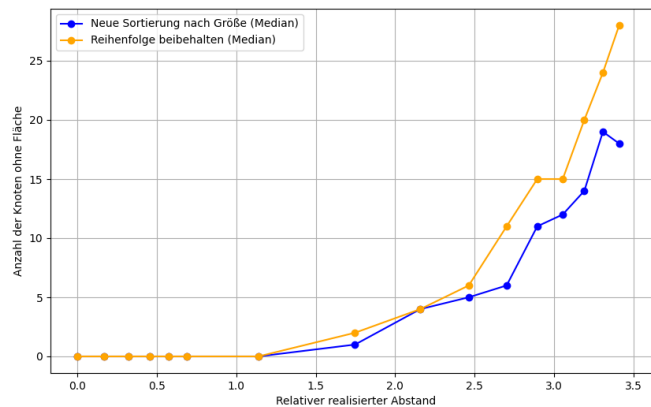


Abbildung 1.37: Anzahl der fehlenden Knoten in Abhängigkeit des relativen Abstands für die zwei verschiedenen Platzierungsstrategien.

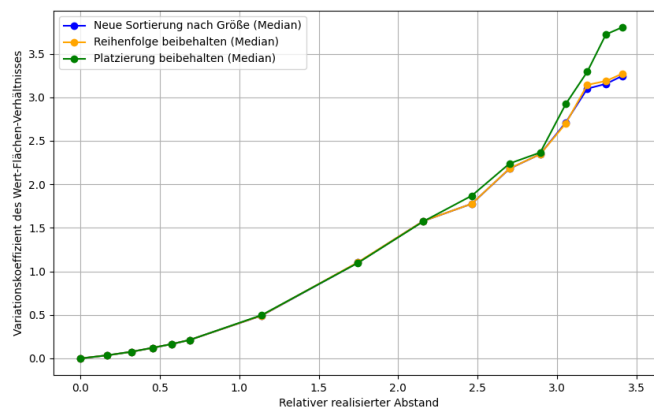


Abbildung 1.38: Varianzkoeffizient des Flächenverhältnisses in Abhängigkeit des relativen Abstands für die drei verschiedenen Platzierungsstrategien.

Die anderen beiden Strategien schneiden deutlich besser ab. In Abbildung 1.37 ist genauer zu erkennen, dass die Strategie, die Knoten neu zu sortieren weniger fehlende Knoten produziert als die Strategie, die Reihenfolge der Knoten beizubehalten.

Anders als erwartet, ist in Abbildung 1.38 zu erkennen, dass alle drei Strategien ähnliche Ergebnisse beim Varianzkoeffizienten des Flächenverhältnisses erzielen.

wie erwartet, ist in Abbildung 1.39 zu erkennen, dass die Strategie, die Knotenplatzierungen bei zu behalten die schlechtesten Seitenverhältnisse produziert. Die anderen beiden Strategien schneiden deutlich besser ab, untereinander verglichen sind sie aber ähnlich.

In Abbildung 1.40 ist zu erkennen, dass alle drei Strategien ähnliche Ergebnisse bei der Platznutzung erzielen.

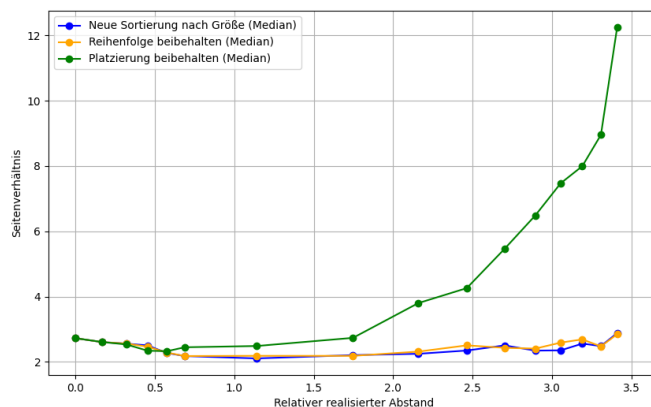


Abbildung 1.39: Durchschnittliches Seitenverhältnis in Abhängigkeit des relativen Abstands für die drei verschiedenen Platzierungsstrategien.

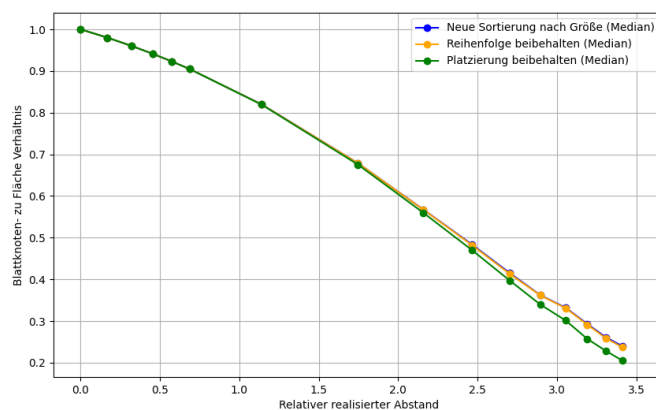


Abbildung 1.40: Platznutzung in Abhängigkeit des relativen Abstands für die drei verschiedenen Platzierungsstrategien. Werte näher bei 1 sind besser.

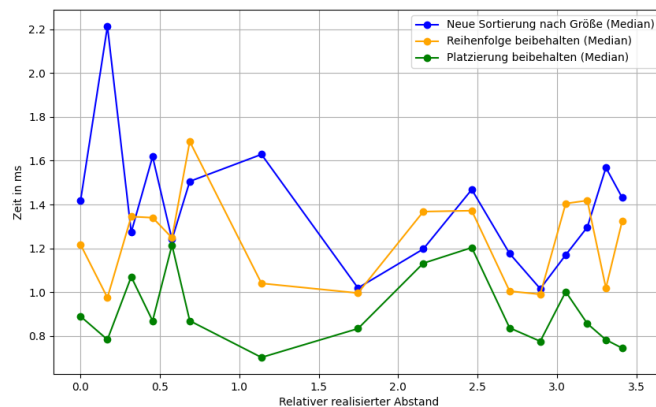


Abbildung 1.41: Zeitaufwand in Abhängigkeit des relativen Abstands für die drei verschiedenen Platzierungsstrategien.

In Abbildung 1.41 wird die Erwartung bestätigt, dass die Strategie, die Knotenplatzierungen beizubehalten am wenigsten Zeit benötigt. Da hier der Squarify Algorithmus nur einmal beim ersten Layoutschritt laufen muss. Die anderen beiden Strategien benötigen mehr Zeit, da hier der Squarify Algorithmus zweimal laufen muss. Die Reihenfolge beizubehalten benötigt minimal weniger Zeit (im Schnitt 0,2ms weniger), da hier nur einmal sortiert werden muss. Dies ist aber eine vernachlässigbare Zeitspanne.

Alles in allem konnten wir zeigen, dass der Ansatz von Lu und Fogarty [LF08], die Knotenplatzierung beizubehalten, für unsere Daten und Kriterien nicht der beste ist. Der Ansatz der neuen Sortierung der Knoten nach Größe schneidet bis auf die Laufzeit in allen Kriterien deutlich besser als Lu und Fogartys Ansatz ab und ist entweder gleich oder etwas besser als der Ansatz, die Reihenfolge der Knoten beizubehalten. Wir entscheiden uns in dieser Arbeit dafür, die Knoten im zweiten Layoutschritt nach Größe zu sortieren, da wir hier die besten Seitenverhältnisse erzielen und die Anzahl der fehlenden Knoten im Vergleich zu den anderen beiden Ansätzen gering bleibt.

1.9 Mehrfache Berechnung

Die Grundlegende Idee dieser Erweiterung ist es, das Layout mehrfach zu berechnen und dabei die Fläche der Knoten immer weiter anzupassen. Warum glauben wir, dass das gut ist. nach dem ersten update schritt ist die vorher-sage der neuen fläche ja oft nicht so gut, und es kann sich noch einiges in der reihenfolge der Knoten ändern oder die Form der Knoten. dadurch ist nach dem zweiten Layout-Schritt die platzierung ja anders, dadurch stimmt die vorhergesagte Fläche der Knoten nicht mehr. dann könnte man nochmal her gehene und die fläche neu berechnen und nochmal den layoutschritt machen. Wir führen dafür einen neuen Größenanpassungsschritt ein. Dieser Schritt ist ähnlich zu dem relativen Größenanpassungsschritt (siehe Listing 2).

Wir stellen also die Hypothese auf, dass durch mehrfaches berechnen des layouts und das aktualisieren der flächen, alle unsere Kriterien, bis auf die laufzeit, besser werden.

Es ist ganz klar in Abbildung 1.42 zu erkennen, zwei Layout-Schritte dauerhaft bessere Ergebnisse erzielen als mehrere Layout-Schritte. Auch alle anderen Kriterien sind entweder gleich oder schlechter. Die Laufzeit ist je nach Anzahl der Layout-Schritte um entsprechenden Faktor höher. Es gibt also keinen Grund, warum die mehrfache Berechnung des Layouts sinnvoll sein sollte. Wir vermuten dass das daran liegt, dass Knoten, die besonders viel Abstand hinzubekommen

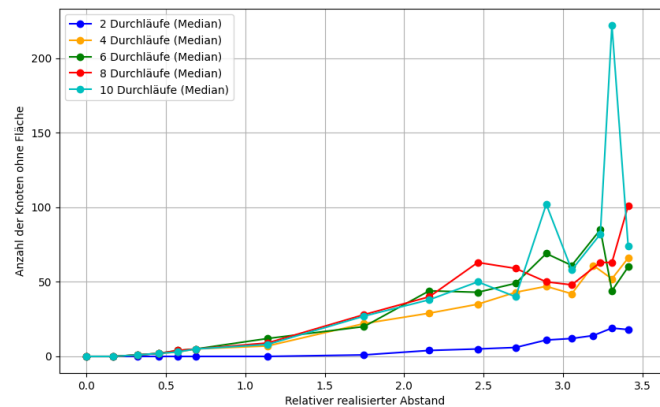


Abbildung 1.42: Anzahl der fehlenden Knoten in Abhängigkeit des relativen Abstands für 2, 4, 6, 8, 10 Layout-Schritte.

1.9.1 Iterative Abstandsvergrößerung

Wir haben soeben gesehen, dass die mehrfache Berechnung des Layouts nicht sinnvoll ist, so wie sie implementiert ist. Wir vermuten, dass das daran liegt,

Die Grundlegende Idee dieser Erweiterung ist es, dass wenn das Layout mehrfach berechnet wird, es besser ist, den Abstand zwischen den Knoten iterativ von Schritt zu Schritt zu vergrößern, anstatt ihn von Anfang an auf den finalen Wert fest zu setzen. Warum gehen wir von dieser Annahme aus? Wenn man im ersten Schritt den Abstand auf den finalen Wert setzt, dann werden die Vorhersagen für die neue Fläche der Knoten im zweiten Schritt nicht exakt sein, das ist ja klar. Umso kleiner die Abstände sind, umso kleiner sind auch die Abweichungen dieser Vorhersagen. Dadurch sollten die Wert-Fläche-Relationen im zweiten Schritt besser sein. Die Annahme ist jetzt, dass wenn man jedes mal den Abstand um einen kleinen Wert vergrößert, dann sind in jedem Schritt die Vorhersagen für die Fläche der Knoten genauer, wodurch die Wert-Fläche-Relationen besser werden. Als Gegenargument könnte man bringen, dass dadurch, dass jedes mal die Abstände größer werden auch jedes mal das Optimale Layout (man kann das optimale layout nicht berechnen, aber wir gehen mal von einem theoretischen optimalen Layout aus) auch in jedem layout schritt anders sein wird. Dadurch wird erst im letzten Layoutschritt das "wahreoptimale Layout angestrebt, wodurch die vorherigen schritte unnötig waren.

Wir überprüfen jetzt die annahme und schauen, welche begründung richtiger ist/welcher effekt mehr gewicht hat.

1.10 Optimierung von Ordnerketten

Bei der Datenanalyse ist aufgefallen, dass es oft vorkommt, dass Ordner nur ein Kind haben bzw. dass es Ketten von Ordnern gibt, die nur ein Ordner als

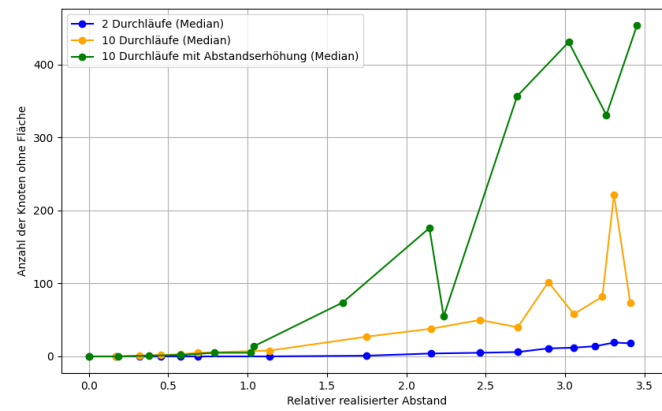


Abbildung 1.43: Anzahl der fehlenden Knoten in Abhängigkeit des relativen Abstands für 2 Layout-Schritte mit direkter Abstandsvergrößerung und 2 Layout-Schritte mit iterativer Abstandsvergrößerung.

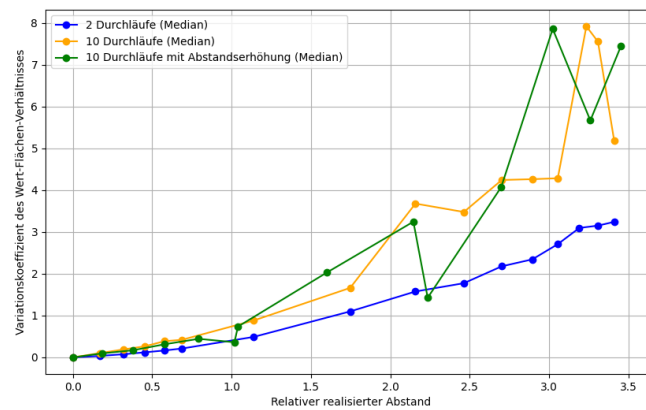


Abbildung 1.44: Varianzkoeffizient des Flächenverhältnisses in Abhängigkeit des relativen Abstands für 2 Layout-Schritte mit direkter Abstandsvergrößerung und 2 Layout-Schritte mit iterativer Abstandsvergrößerung.

Kind haben. Häufig sind diese Ketten auch nicht nur einen Ordner tief, was gar nicht so viel Abstand bzw. Platz verbrauchen würde, doch es gibt auch einige Ketten die 4 Ordner tief sind, was dann schon wieder (mehr) als viel mal so viel Platz für Abstände aufbraucht durch die Verschachtelung. Obwohl im Grunde diese Schachtelung keine wirklichen Strukturinformationen enthält. Viele IDEs wie zum Beispiel auch IntelliJ IDEA oder VSCode verbergen diese Ordnerstruktur zum Beispiel auch automatisch, indem solche Ordnerketten nur als ein Ordner mit entsprechendem zusammengeführten Namen (in dem Fall *main/kotlin/de/maibornwolff/codecharta/analysers/analyserinterface*) angezeigt wird. Diese Optimierung kann auch für unsere Visualisierung übernommen werden, indem solche Ordnerketten zu einem Ordner zusammengefasst werden.

Da dies nicht nur eine algorithmische Änderung ist, sondern auch einen direkten Einfluss auf die Visualisierung hat, werden wir nicht nur die Auswirkungen auf die messbaren Kriterien untersuchen, sondern auch kurz diskutieren, welche Auswirkungen dies auf die generellen Kriterien von Visualisierungen hat, wie wir sie in Abschnitt ?? definiert haben. Es ergeben sich folgende Auswirkungen auf die Ziele der Visualisierung:

- **Informationsgehalt und effiziente Nutzung des Platzes:** Wird verbessert, da weniger Platz für Abstände zwischen den Ordnern benötigt wird und weniger Knoten im Layout sind.
- **Niedrige visuelle Komplexität und Abstraktion:** Wird verbessert, da Codebasen mit tiefen Ordnerstrukturen einfacher dargestellt werden können.
- **Skalierbarkeit:** Wird verbessert, da weniger Knoten im Layout sind und große Codebasen mit tiefen Ordnerstrukturen einfacher dargestellt werden können.
- **Visuelle Metaphern:** Dies hat keine Auswirkungen auf die visuelle Metapher der Stadt.

Wir würden sagen, dass die Vorteile dieser Änderung die Nachteile stark überwiegen.

Im Schnitt werden ca. 175 Ordner pro Codebasis gefaltet (12 Median). Bei ca. 958 Knoten (Median 242) sind das ca. 18% (bzw. 5% Median) der Knoten, die durch die Faltung verschwinden. Also schon einige.

Folgend kommen die Daten:

Die Daten zeigen in allen Aspekten eine Verbesserung durch das Falten von Ordnerketten. Wir sprechen uns also dafür aus, dass Ordnerketten immer gefaltet werden sollten, von den messbaren Kriterien her sehen wir keine Gründe, warum das nicht gemacht werden sollte. Das einzige Ziel, was dagegen spricht, ist die Korrelation mit dem Code, die dadurch verschlechtert wird. Alles in allem sprechen wir uns also dafür aus und werden dies fix für unseren Algorithmus implementieren.

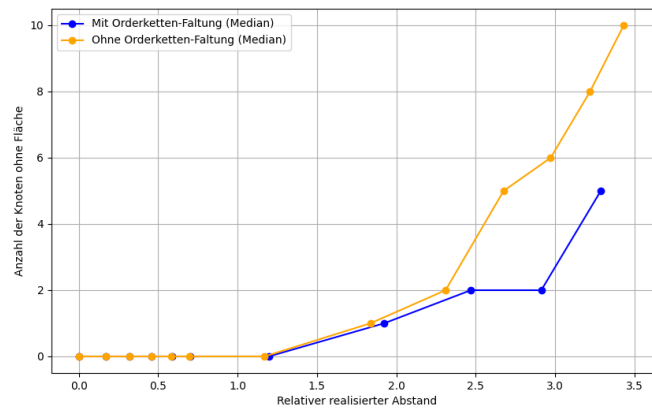


Abbildung 1.45: Anzahl der fehlenden Knoten in Abhängigkeit des relativen Abstands für gefaltete und ungefaltete Ordnerketten.

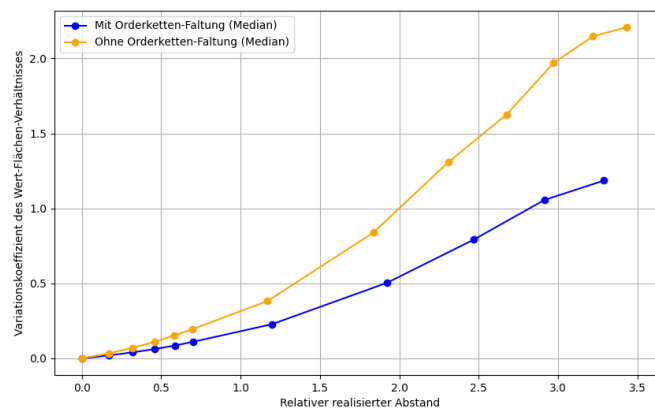


Abbildung 1.46: Varianzkoeffizient des Flächenverhältnisses in Abhängigkeit des relativen Abstands für gefaltete und ungefaltete Ordnerketten.

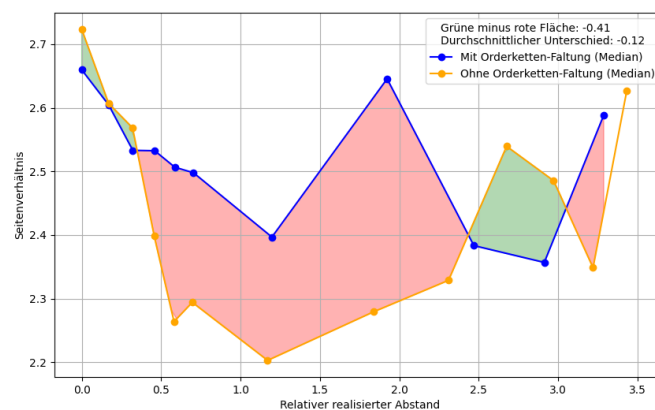


Abbildung 1.47: Durchschnittliches Seitenverhältnis in Abhängigkeit des relativen Abstands für gefaltete und ungefaltete Ordnerketten.

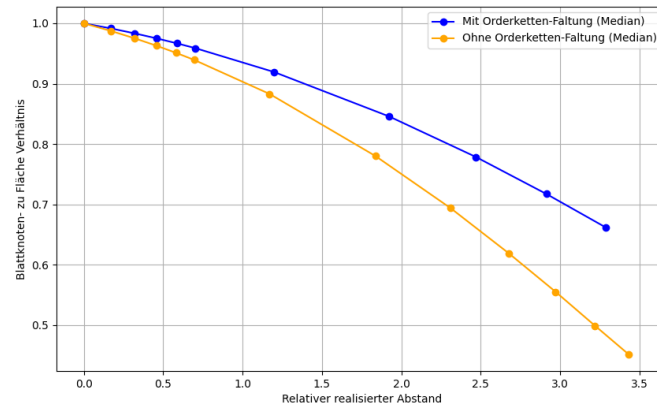


Abbildung 1.48: Platznutzung in Abhängigkeit des relativen Abstands für gefaltete und ungefaltete Ordnerketten. Werte näher bei 1 sind besser.

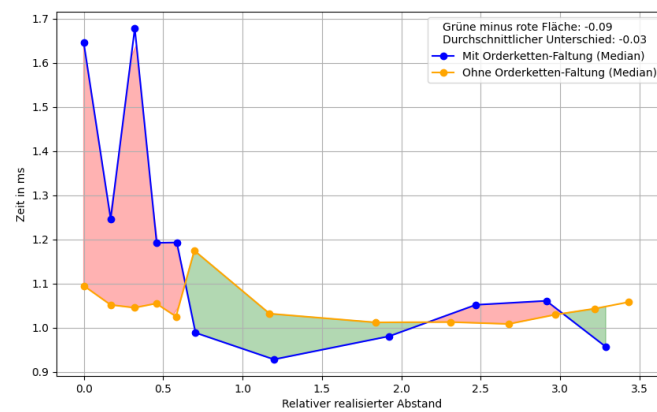


Abbildung 1.49: Zeitaufwand in Abhängigkeit des relativen Abstands für gefaltete und ungefaltete Ordnerketten.

1.11 *Abstände zwischen Geschwister Knoten*

Abstände zwischen Geschwister Knoten sind eine wichtige Entscheidung. Im Ursprünglichen Squarify Algorithmus Paper [BHVWoo] wird der Nested Ansatz mit Abständen vorgestellt, jedoch ohne Abstände zwischen Geschwistern. Viele Tools, verwenden jedoch Abstände auch zwischen Geschwisterknoten [WLo7; Gmbc; Gmba]. Was sind die Vor- und Nachteile von Abständen zwischen Geschwistern?

- **Informationsgehalt und effiziente Nutzung des Platzes:** Wird verschlechtert, da jeder Knoten mit geschwistern mehr Platz benötigt.
- **Niedrige visuelle Komplexität und hohe Verständlichkeit:** Wird verbessert, da die Knoten besser voneinander getrennt werden und somit die Struktur besser erkennbar ist. Allerdings muss das nicht der Fall sein, man könnte zum Beispiel die Kanten der Knoten mit einer anderen Farbe einfärben, um die Knoten besser voneinander zu trennen.
- **Skalierbarkeit:** Eigentlich wenig auswirkung auf skalierbarkeit, weder die laufzeit ändert sich signifikant (in unsrem algorithmus werden die abstände erzeugt, indem am ende alle knoten um den abstand verkleinert werden - also im grunde kommt laufzeit von N hinzu, wo N die Anzahl der Knoten ist, das ist aber nicht ausschlaggebend). Also wenn wird die Skalierbarkeit minimal schlechter.
- **Korrelation mit dem Code:** Keine auswirkung
- **Stabilität gegenüber Änderungen:** Auch nicht wirklich

Die Anpassung, die Knoten mit Kanten zu versehen, natürlich stark abhängig von der gewählten Farbgebung (auch Schattierung, Struktur oder Transparenz). Wie in der Problemstellung definiert (siehe Abschnitt ??) sollte das Layout so gestaltet sein, dass es möglich ist, weitere Metriken durch Farbgebung zu visualisieren. Die Kanten widersprechen dieser Einschränkung nur in Ausnahme fällen. Die meisten Visualisierungen in der Praxis verwenden einfache Einfärbungen [WLo7; Gmbc; Gmba], was bedeutet, dass man einfach Schwarz oder eine Komplementärfarbe als Kantenfarbe nutzen kann, ohne dass es zu Konflikten mit der Farbgebung kommt. Die einzige Ausnahme, die mir bekannt ist, ist wäre die Nutzung von Linien, Schachbrettmuster oder ähnlichen Strukturen, durch die eine Kante nicht mehr als solche erkennbar wäre. Dies könnte zum Beispiel bei dem Ansatz aus dem paper *E-Quality* von Ural Erdemir et al. [ETB11] der Fall sein, da hier zum Beispiel ein Gittermuster verwendet wird, um einen Mangel an Kohäsion zu visualisieren, eine Kante, selbst in anderer Farbe, würde hier eher stören oder in dem Muster untergehen.

1.12 *Beschriftung der Knoten*

Die Beschriftung der Knoten ist ein wichtiger Aspekt der Visualisierung, da sie es dem Nutzer ermöglicht, die Knoten zu identifizieren und zu verstehen,

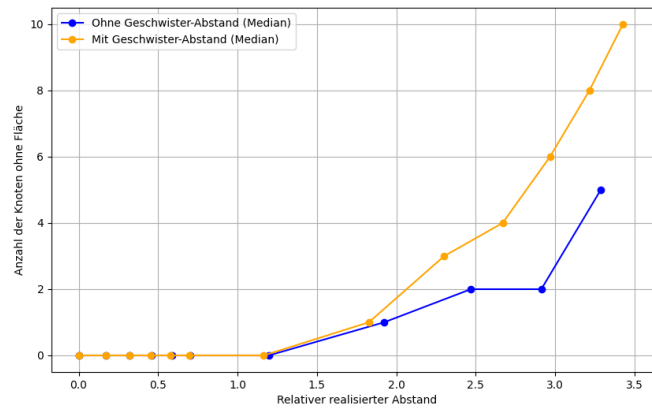


Abbildung 1.50: Anzahl der fehlenden Knoten in Abhängigkeit des relativen Abstands für Abstände zwischen Geschwistern und keine Abstände zwischen Geschwistern.

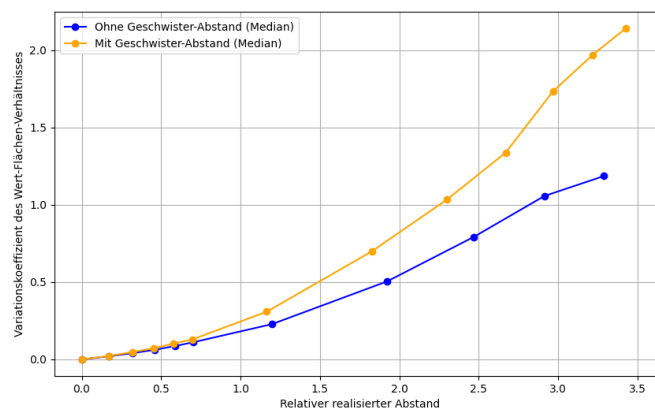


Abbildung 1.51: Varianzkoeffizient des Flächenverhältnisses in Abhängigkeit des relativen Abstands für Abstände zwischen Geschwistern und keine Abstände zwischen Geschwistern.

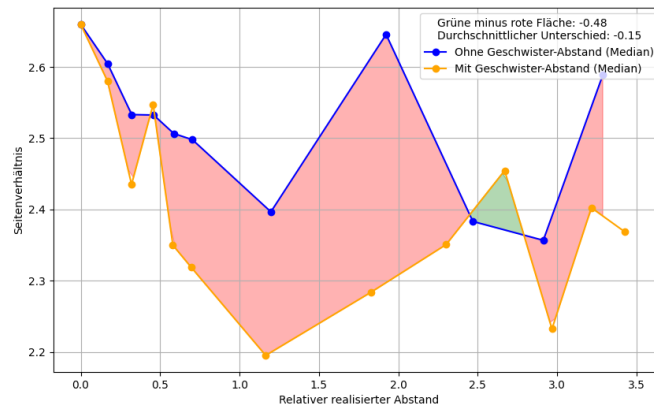


Abbildung 1.52: Durchschnittliches Seitenverhältnis in Abhängigkeit des relativen Abstands für Abstände zwischen Geschwistern und keine Abstände zwischen Geschwistern.

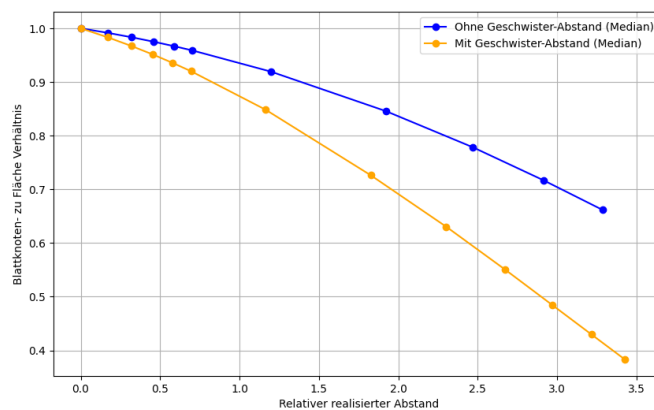


Abbildung 1.53: Platznutzung in Abhängigkeit des relativen Abstands für Abstände zwischen Geschwistern und keine Abstände zwischen Geschwistern. Werte näher bei 1 sind besser.

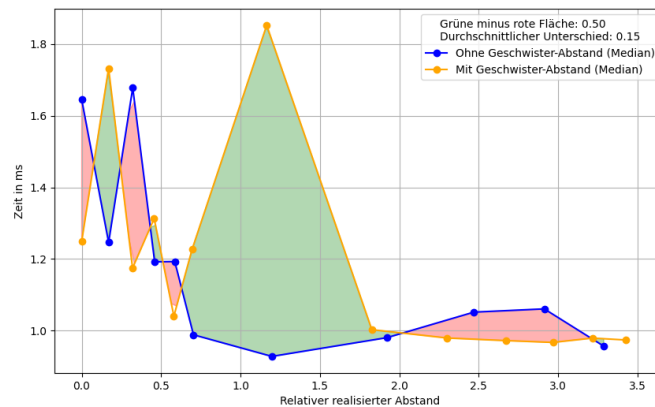


Abbildung 1.54: Zeitaufwand in Abhängigkeit des relativen Abstands für Abstände zwischen Geschwistern und keine Abstände zwischen Geschwistern.

welche Informationen sie repräsentieren. Es gibt verschiedene Ansätze, um die Beschriftung der Knoten zu gestalten. Jeder Ansatz hat seine Vor- und Nachteile. Ein häufig verwendeter Ansatz ist wie zum Beispiel auch beim in Abschnitt 1.1.2 vorgestellten Cascaded Treemap Ansatz [LFo8]. Ordner erhalten eine zusätzliche horizontale Fläche, entweder unten oder oben, die den Namen des Ordners enthält. Dies nimmt allerdings viel Platz ein und erzeugt die selben Schwierigkeiten für den Algorithmus, wie die Abstände zwischen den Knoten. Außerdem kann es passieren, dass die Beschriftung zu lang für die Breite der Knotens ist, wodurch die Beschriftung entweder abgeschnitten wird oder über den Knoten hinauswächst. Hao Lü and James Fogarty schlagen vor dynamisch zu entscheiden, ob die Beschriftung der Knoten angezeigt werden soll oder nicht [LFo8]. Wenn das Hinzufügen der Beschriftung den Knoten dazu führen würde, dass gar kein Platz mehr für den Knoten selbst übrig bleibt, wird die Beschriftung nicht angezeigt. Der Vorteil dieses Ansatzes ist, dass sicher gegangen wird, dass durch die Beschriftung kein Knoten verschwindet. Der Nachteil ist, dass selbst wenn die Änderung der Knotenfläche zum Knotenwert durch das Hinzufügen der Beschriftung riesig wird (wenn die Fläche also super klein wird), die Beschriftung trotzdem hinzugefügt wird.

Offensichtlich wird das Hinzufügen von Beschriftungen alle unsere Kriterien verschlechtern.

Wir schlagen einen statischen Ansatz vor, bei dem nur die Beschriftungen der Top N Ordner angezeigt werden. Die Wahl von N ist nun die Stellschraube für die nicht für jede Codebasis gleich ist. Es ist Stellschraube um zwischen Informationsgehalt und visueller Komplexität zu balancieren.

- Je höher N ist, desto mehr Informationen (also Ordnernamen) werden angezeigt, was den Informationsgehalt erhöht.

- Je niedriger N ist, desto weniger Informationen werden angezeigt, was die visuelle Komplexität reduziert, viele Beschriftungen können schnell ablenken und die Visualisierung unübersichtlich machen.
- Außerdem wird die Korrelation mit dem Code verbessert, da ein Übertragen von Visualisierung zur Codebasis einfacher wird mit höherem N .
- Skalierbarkeit hängt nicht ab von N , da N ja nicht mitwächst, sondern statisch ist, ob noch mehr Order und noch mehr Tiefe hinzu kommt, macht keinen wirklichen Unterschied.
- Stabilität gegen Änderungen wenn überhaupt sinkt es, weil Änderung vom Namen z.B. auf einmal sichtbar wird (das ist natürlich gewollt, trotzdem in unserer Messung hier erstmal negativ - wenn auch vernachlässigbar) - eventuell werden dadurch in zwei Versionen die selben Knoten, die jetzt anders heißen, nicht mehr so einfach als gleich identifiziert, obwohl diese die selbe Form haben und ohne die Beschriftung vielleicht anhand der Form als gleich identifiziert werden würden - zugegeben sehr theoretisch und wahrscheinlich nicht relevant aber trotzdem erwähnenswert.

Mit diesem Ansatz hat man natürlich immernoch die gleichen Probleme wie eingangs beschrieben, allerdings sind diese abhängig von dem Wert N und außerdem werden diese minimiert, da wirklich nur die Top Knoten beschriftet werden, wodurch die Fläche der unteren Knoten trotzdem besser vorhergesagt werden kann und sich dieser Fehler nicht nach oben propagiert.

1.13 Fazit

Wir haben in diesem Kapitel verschiedene Erweiterungen und Anpassungen des Squarify Algorithmus für unsere Anforderungen vorgestellt und evaluiert. Dabei haben wir verschiedene bestehende Ansätze aus der Literatur getestet. Der Cascaded Algorithmus von Lu und Fogarty [LF08] hat dabei einige Ideen gegeben. Ihr Ansatz am besten mit dem Absoluten Größenanpassung mit Skalierung und festen Knotenplatzierungen zu vergleichen. Dadurch dass die Autoren ihren Ansatz nicht speziell für Softwarequalitätsmetriken optimiert haben, konnten wir für einige Ideen zeigen, dass sie auch für unsere Anforderungen gut funktionieren und für andere, dass unsere Ideen besser funktionieren. Der einzige Ansatz der glaube ich nicht weiter verfolgt werden muss ist die Mehrfache Berechnung des Layouts, ich hatte nicht das Gefühl, dass man hier noch Anpassungen an dieser Idee vornehmen könnte, die zu besseren Ergebnissen führen. Außerdem können die verwendeten Ansätze selbst noch weiter verbessert werden. Die hier verwendeten Ansätze sind zum Großteil komplett neu und in noch keiner anderen Arbeit vorgestellt. Diese Ansätze sind V1 Versionen und können sicherlich noch weiter verbessert werden.

Das Treemap problem ist durch unseren Ansatz stark verbessert worden, allerdings ist es immernoch ein NP-Hartes Problem, weshalb es keine perfekte Lösung gibt. Es gibt sicherlich noch unzählige Ideen, um unseren Ansatz zu verbessern, wie zum Beispiel: Anstatt feste Abstände. Dynamische Abstände die sich an Tiefe oder Anzahl der Geschwister orient. Oder andere Größenanpassungen, die auf anderen Heuristiken basieren. Oder andere Sortier- und Platzierungsstrategien. Oder andere Ansätze um die Knoten zu beschriften. Oder oder oder...

Alles in allem Empfehlen wir folgende Einstellungen für den Algorithmus:

- **Abstände zwischen Geschwistern:** Manuelle Wahl zwischen 0,5% und 3% relativem Abstand je nach Software und Ziel der Visualisierung (siehe Abschnitt [1.11](#))
- **Angestrebtes Seitenverhältnis:** 1,05 (siehe Abschnitt [1.5](#))
- **Sortierung der Knoten:** Absteigend nach Größe (siehe Abschnitt [1.8](#))
- **Platzierung der Knoten nach erstem Layoutschritt:** Neue Sortierung absteigend nach Größe (siehe Abschnitt [1.8](#))
- **Mehrfache Berechnung:** Nein (siehe Abschnitt [1.9](#))
- **Optimierung von Ordnerketten:** Ja (siehe Abschnitt [1.10](#))
- **Beschriftung der Knoten:** Manuelle Wahl zwischen x und y (siehe Abschnitt [1.12](#))

Wir werden exakt diese Einstellungen verwenden, um die Visualisierungen für die Umfrage im nächsten Kapitel zu erstellen.

UMFRAGE

Dieses Kapitel beschreibt die Umfrage, die mit dem Ziel durchgeführt wurde, die für diese Arbeit implementierten Visualisierungen zu vergleichen und zu bestimmen, welche Visualisierung sich für den zuvor definierten Anwendungsfall am besten eignet (siehe Abschnitt ??). Zunächst wird der Aufbau der Umfrage beschrieben und begründet (Abschnitt 2.1). Anschließend werden die Ergebnisse der Umfrage präsentiert und diskutiert (Abschnitt 2.2).

2.1 *Ablauf der Umfrage*

Das Ziel der Umfrage besteht darin, zu ermitteln, wie gut die verschiedenen Visualisierungen geeignet sind, die Nachvollziehbarkeit von Expertenanalysen zur Softwarequalität zu unterstützen. Die vollständige Umfrage ist im Anhang dokumentiert (siehe Anhang REFERENZ HIER). Die Umfrage wurde mit dem Online-Tool *SurveyMars* [Sur] erstellt und durchgeführt. Die Umfrage ist in vier Abschnitte unterteilt:

1. Einleitung und Vorwissen (Abschnitt 2.1)
2. Vergleich anhand von Szenarien (Abschnitt 2.1)
3. Direkte Bewertung der Visualisierungen (Abschnitt 2.1)
4. Offene Fragen (Abschnitt 2.1)

Einleitung

Die Umfrage beginnt mit einer kurzen Einleitung und einer Aufklärung über Zielsetzung, Rahmen und Datenschutz. Anschließend wird erhoben, ob die befragte Person mit Softwareentwicklung vertraut ist und ob sie mit Software-Visualisierungen vertraut ist. Diese Fragen dienen zum einen dazu, Erklärungen an das Vorwissen der Befragten anzupassen, und zum anderen dazu, die Ergebnisse später besser interpretieren zu können. Darauf folgt, abhängig vom angegebenen Vorwissen, eine kurze Einführung in die Themen Software, Softwarequalität und Software-Visualisierungen. Software-Visualisierungen werden exemplarisch anhand einer 2.5D-Voronoi-Visualisierung erläutert, um den Befragten ein Grundverständnis für die Interpretation von Software-Visualisierungen zu vermitteln.

2. Vergleich anhand von Szenarien

Die Umfrage orientiert sich eng an in der Praxis durchgeführten Software-Audits und Health-Checks, wie sie beispielsweise von der *MaibornWolff GmbH*

oder anderen Softwareentwicklern durchgeführt werden. Ziel solcher Analysen ist es, die Qualität einer Software zu bewerten, Schwachstellen zu identifizieren und darauf aufbauend Empfehlungen zur Verbesserung der Softwarequalität zu formulieren. Visualisierungen dienen in diesem Kontext dazu, Analyseergebnisse nachvollziehbar zu kommunizieren und valide zu begründen. Da entsprechende Praxisanalysen vertrauliche Daten enthalten, kann an dieser Stelle kein reales Beispiel gezeigt werden. Die für die Umfrage verwendete Codebasis stammt daher nicht aus einem tatsächlichen Audit, die erstellten Visualisierungen orientieren sich jedoch eng an realen Darstellungen. Als Codebasis für die Visualisierungen wird das Open-Source-Projekt *CodeCharta* verwendet, da es mit circa 1200 Dateien eine mittlere Größe aufweist [Gmbb].

Den Befragten werden sieben Softwareanalysen in Textform präsentiert, die jeweils ein spezifisches Szenario adressieren. In der ersten Analyse geht es beispielsweise um den Fall, dass ein einzelner Entwickler einen großen Teil des Codes geschrieben hat. Zu jedem Szenario werden sechs Visualisierungen erstellt. Fünf davon sind die in dieser Arbeit implementierten Visualisierungen, die dieselbe Codebasis und dieselben Metriken darstellen. Die sechste Visualisierung stammt ebenfalls aus diesem Set, basiert jedoch auf einer anderen Codebasis oder abweichenden Metriken, sodass diese nicht zur jeweiligen Analyse passt. Diese absichtlich nicht passende Visualisierung dient als Plausibilitätskontrolle, um zu prüfen, ob die Befragten die Visualisierungen tatsächlich mit der Analyse abgleichen und nicht lediglich eine visuelle Präferenz angeben.

Für jede der sechs Visualisierungen wird erhoben, inwieweit sie die Analyse nachvollziehbar macht. Zur Begrenzung der Umfragedauer werden pro Szenario drei der sechs Visualisierungen randomisiert angezeigt, sodass jede befragte Person insgesamt 21 der insgesamt 42 Visualisierungen sieht.

Die Analyse beziehen sich sowohl auf die Struktur als auch auf die Qualitätsmetriken einer Software. Aus diesem Grund soll sichergestellt werden, dass beide Aspekte von den Betrachtenden in der Visualisierung erkannt und nachvollzogen werden können. Von den sieben Szenarien zielen daher drei primär auf Metriken, drei auf die Struktur und eines auf die Kombination beider Aspekte ab:

- **Szenarien 1 bis 3** fokussieren sich auf Metriken. Hier geht es um das Auffinden von Hotspots, um die Frage, ob Probleme verteilt, vereinzelt oder gebündelt auftreten, sowie um das Verknüpfen unterschiedlicher Metriken.
- **Szenarien 4 bis 6** adressieren Strukturprobleme. Beispiele sind breite Strukturen, unübersichtliche Strukturen oder sehr tiefe Strukturen.
- **Szenario 7** kombiniert beide Aspekte der Softwareanalyse. Beispielsweise wird die Erkennung von Duplikationen, die sich dadurch auszeichnen, dass sowohl Struktur als auch Metriken der Software übereinstimmen.

Die Szenarien sind aufsteigend in der Schwierigkeit gestaltet. Szenario 1 ist vergleichsweise offensichtlich, während Szenario 3 anspruchsvoller ist. Entsprechendes gilt für die Szenarien 4 bis 6. Szenario 7 stellt eine weitere Steigerung dar, da hier die gleichzeitige Berücksichtigung von Struktur und mehreren Metriken erforderlich ist.

Im Folgenden wird Szenario 3 exemplarisch dargestellt. In diesem Szenario soll ein Software-Hotspot mit komplexen, große und ungetesteten Dateien identifiziert werden. Im Beispielbild ist der Ordner „Packaging“ blau markiert. Es finden sich darin zwei bis drei Dateien, die eine große Fläche aufweisen, hoch sind und rot eingefärbt sind (siehe Abbildung 2.1). Dem Betrachtenden soll deutlich werden, dass die Kombination ungünstiger Werte ausschließlich in diesem Ordner auftritt. Der Ordner stellt somit einen Hotspot dar, der genauer untersucht werden sollte. Gleichzeitig ist zum Beispiel erkennbar, dass andere Dateien, die ebenfalls groß und hoch sind, nicht rot sind, also eine bessere Testabdeckung aufweisen.

Beispiel Szenario: Kombinieren von drei Metriken

Oft reicht es nicht aus, nur eine einzelne Metrik zu betrachten. Hohe Komplexität ist manchmal akzeptabel, wenn die Dateien dafür gut getestet sind. Tests sorgen dafür, dass Fehler früh erkannt werden und die Wartung leichter fällt. Die Testabdeckung zeigt, wie viel Prozent des Codes durch Tests ausgeführt werden.

Die Analyse zeigt, dass manche komplexe Dateien - speziell die großen - gut getestet sind, was positiv ist. Hier besteht kein Handlungsbedarf. Es gibt jedoch Ausnahmen: Im Ordner "Packaging" (blau markiert) liegen viele ungetestete Dateien, von denen einige groß und sehr komplex sind.

Legende: Fläche = Anzahl der Codezeilen Höhe = Komplexität (niedrig=einfach, hoch=komplex) Farbe = Testabdeckung in Prozent (0=rot, über 100=grün) Ordner sind grau, Dateien sind farbig- von rot zu gelb zu grün

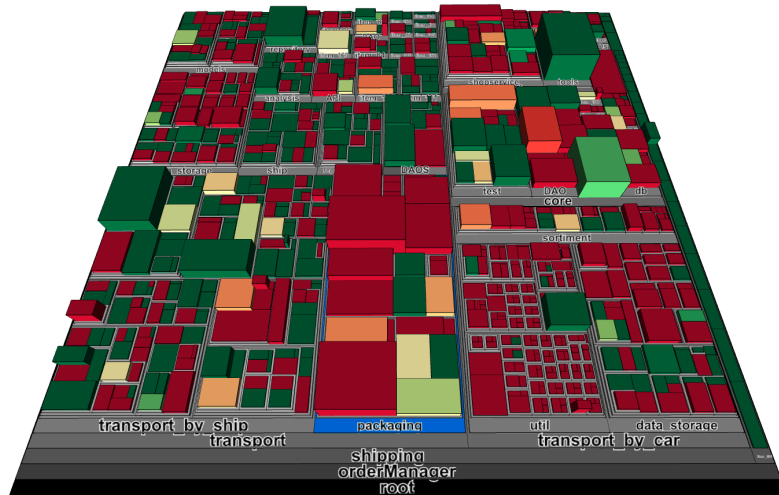


Abbildung 2.1: Beispiel Visualisierung für das Szenario *Kombinieren von drei Metriken* (Treemap).

Wie sehr hilft dir die Visualisierung, die Analyse der Experten nachzuvollziehen?

3. Direkte Bewertung der Visualisierungen

In diesem Teil der Umfrage führen die Befragten jene Bewertungen durch, die in Abschnitt ?? vorgenommen wurden, um zu prüfen, ob andere Personen mit Erfahrung in Software-Visualisierungen zu ähnlichen Einschätzungen gelangen (siehe Abschnitt ??). Aus diesem Grund werden ausschließlich Personen berücksichtigt, die zu Beginn angegeben haben, bereits Erfahrung mit Software-Visualisierungen zu besitzen.

Für jede der fünf Visualisierungen werden die Befragten, analog zur Literaturrecherche, gebeten, für jedes Kriterium eine Bewertung auf einer Skala von 1 bis 5 abzugeben. Dazu wird jedes Kriterium kurz erläutert und anschließend die Bewertung pro Visualisierung erhoben.

4. Offene Fragen

Am Ende der Umfrage folgen drei offene Fragen, um zusätzliches Feedback zu den Visualisierungen und zur Umfrage zu erhalten:

1. **Persönliches Fazit zu Software-Visualisierungen:** Findest Du nach Deiner Erfahrung mit den verschiedenen Visualisierungen Software-Visualisierungen generell sinnvoll und nützlich? Warum/warum nicht?
2. **Konkrete Anmerkungen zu einzelnen Visualisierungen:** Hast Du konkrete Anmerkungen zu einzelnen Visualisierungen? Gab es etwas, das Dir besonders gut oder besonders schlecht gefallen hat?
3. **Kritik, Anmerkungen zu Dingen, die nicht abgefragt wurden:** Hast Du Kritik oder Anmerkungen zu Dingen, die in der Umfrage nicht abgefragt wurden?

Szenario	Name	$\bar{O}$ -Passend	σ -Passend	$\bar{O}$ -Nicht passend	σ -Nicht passend	Unterschied
1	Treemap	4.5	0.7	2.4	1.6	2.0
2	Streetmap	3.4	1.2	2.8	1.8	0.6
3	Circular Treemap	3.7	1.2	2.4	1.3	1.3
4	Treemap	4.2	1.2	3.1	1.4	1.1
5	Sunburst	3.9	1.2	3.3	1.4	0.6
6	Treemap	3.7	1.1	1.6	1.3	2.2
7	Code City	3.8	1.5	3.3	1.4	0.5
Durchschnitt		3.9	1.2	2.7	1.5	1.2

Tabelle 2.1: Unterschied in der Bewertung von passenden und nicht passenden Visualisierungen pro Szenario. $\bar{O}$ = Durchschnitt, σ = Standardabweichung.

2.2 Ergebnisse der Umfrage

In diesem Abschnitt werden die Ergebnisse der Umfrage vorgestellt und diskutiert. Insgesamt nahmen 20 Personen teil. Davon verfügten 14 Personen bereits über Erfahrung mit Softwareentwicklung (7 Informatikstudierende, 6 Softwareentwickler, 1 Auftraggeber/Manager o.Ä.). Von diesen 14 Personen mit Erfahrung in der Softwareentwicklung hatten 8 Personen bereits Erfahrung mit Software-Visualisierungen, davon 4 als Nutzende und 4 als Entwickelnde solcher Visualisierungen. Diese Gruppe von 8 Personen wird im Folgenden als **Visualisierungsexperten** bezeichnet.

2.2.1 Bewertung anhand von Szenarien

Im Mittel wurde jede Visualisierung in jedem Szenario von 10 Personen bewertet. Die Spannweite reichte von 5 bis 16 Bewertungen pro Visualisierung und Szenario. Die Ergebnisse sollten jedoch mit Vorsicht interpretiert werden, da die Stichprobengröße pro Visualisierung und Szenario relativ klein ist.

Zunächst wird überprüft, ob die als nicht passend identifizierten Visualisierungen tatsächlich schlechter bewertet wurden als die als passend eingestuft. Tabelle 2.1 zeigt, dass die nicht passenden Visualisierungen im Mittel um 1,2 Punkte schlechter bewertet wurden als die passenden Visualisierungen. Dies spricht dafür, dass die Befragten die Visualisierungen tatsächlich mit der Analyse abgeglichen haben und nicht lediglich eine visuelle Präferenz ausdrückten. Da die Standardabweichung in beiden Gruppen jedoch ähnlich hoch oder höher ist als der durchschnittliche Unterschied, liegt nahe, dass die Unterscheidung zwischen passenden und nicht passenden Visualisierungen vielen Befragten schwerfiel oder teilweise gar nicht wahrgenommen wurde. In verschiedenen Szenarien und bei verschiedenen Visualisierungen wurden die nicht passenden Visualisierungen unterschiedlich stark erkannt. Auffallend groß ist der Erkennungswert bei Treemaps in den Szenarien 1 und 6. Positive Differenzen zeigen sich zudem bei der Circular Treemap im Szenario 3 sowie bei Treemap im Szenario 4. Dies legt nahe,

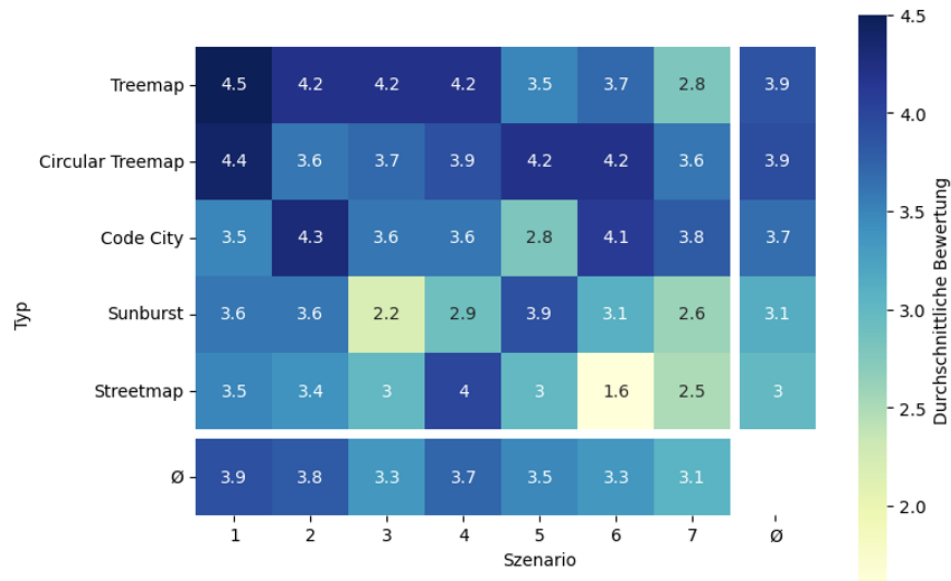


Abbildung 2.2: Durchschnittliche Bewertung der passenden Visualisierungen pro Szenario.

dass bei Treemap und Circular Treemap besonders gut erkennbar ist, wenn die Analyse fehlerhaft ist.

In Abbildung 2.2 sind die durchschnittlichen Bewertung der passenden Visualisierungen pro Szenario dargestellt. Es lassen sich mehrere Punkte aus der Grafik ablesen. Erstens fallen die Bewertungen in den vergleichsweise einfachen Szenarien 1 und 4 erwartungsgemäß besser aus als in den anspruchsvolleren Szenarien 3, 6 und 7. Außerdem wird sichtbar, wie die verschiedenen Visualisierungen in den einzelnen Szenarien abschneiden. Treemap und Circular Treemap erreichen im Mittel die besten Werte, mit nahezu einem Punkt Vorsprung gegenüber der schwächsten Visualisierung Streetmap.

Die durchschnittliche Bewertung über alle passenden Visualisierungen und Szenarien beträgt 3,5 Punkte. Auffällig ist, dass die Treemap in allen Szenarien gleichauf oder deutlich über dem Durchschnitt von 3,5 liegt. Eine Ausnahme bildet Szenario 7, in dem die Treemap deutlich schlechter abschneidet als beispielsweise Code City. Ein möglicher Grund ist die fehlende Stabilität des Treemap-Algorithmus. Der Code-City-Algorithmus rendert aufgrund seines Bottom-up-Ansatzes gleiche Strukturen immer gleich, auch wenn diese in verschiedenen Ordnern liegen. Beim Treemap-Algorithmus ist dies nicht gewährleistet. Gleiche Strukturen können unterschiedlich gelayoutet werden, da der Elternknoten möglicherweise eine andere Größe und/oder ein anderes Seitenverhältnis aufweist.

Abschließend wird betrachtet, welche Visualisierung für welches Ziel am besten geeignet ist. Zur Erinnerung: Szenario 1 bis 3 zielen auf die Erkennung von Metriken ab, Szenario 4 bis 6 auf die Erkennung von Strukturproblemen, und Szenario 7 kombiniert beide Ziele.



Abbildung 2.3: Durchschnittliche Bewertung der Visualisierungen pro Zielgruppe. Metriken = Szenario 1 bis 3, Struktur = Szenario 4 bis 6, Beides = Szenario 7.

Abbildung 2.3 zeigt, dass Treemaps am besten für die Erkennung von Metriken geeignet sind. Circular Treemap und Code City schneiden hierfür ebenfalls vergleichsweise gut ab. Für die Erkennung von Strukturproblemen erzielt die Circular Treemap die besten Ergebnisse, gefolgt von Treemap und Code City.

2.2.2 Direkte Bewertung der Visualisierungen

In diesem Abschnitt werden die Ergebnisse der direkten Bewertung der Visualisierungen dargestellt. Diese direkte Bewertung wurde ausschließlich von den Visualisierungsexperten durchgeführt. Ziel ist es zu prüfen, ob die Bewertungen aus der Literaturrecherche auch von anderen erfahrenen Personen geteilt werden. Es wurden 8 Personen befragt. Die Standardabweichung der Expertenbewertungen liegt im Mittel bei einem Punkt. Bei einer Skala von 1 bis 5 und einer Stichprobengröße von 8 sind die Ergebnisse daher eher als Tendenz zu interpretieren.

Insgesamt bewerten die Experten etwas strenger als in der Literaturrecherche dieser Arbeit. Im Mittel vergeben sie 0,52 Punkte weniger. Auffällig ist das Kriterium der visuellen Komplexität, das von den Experten deutlich schlechter eingeschätzt wird als in der Literaturrecherche. Eine mögliche Erklärung ist, dass in der Literaturrecherche auch Visualisierungen betrachtet wurden, die wesentlich komplexer waren als die hier verwendeten, wodurch

Name	Informationsgehalt			Visuelle Komplexität			Skalierbarkeit			Metapher			Gesamtwertung		
	U	L	U-L	U	L	U-L	U	L	U-L	U	L	U-L	U	L	U-L
Treemap	4.3	4	0.3	3.8	5	-1.2	3.7	4	-0.3	3.5	3	0.5	3.8	4.0	-0.2
Code City	2.8	3	-0.2	4.2	5	-0.8	3.0	3	0.0	3.7	4	-0.3	3.4	3.8	-0.4
Sunburst	3.3	3	0.3	3.0	5	-2.0	2.2	4	-1.8	3.3	2	1.3	3.0	3.5	-0.5
Circular Treemap	2.7	3	-0.3	3.3	5	-1.7	2.8	3	-0.2	2.8	2	0.8	2.9	3.2	-0.3
Streetmap	2.2	3	-0.8	2.3	5	-2.7	2.7	3	-0.3	3.2	4	-0.8	2.6	3.8	-1.2

Tabelle 2.2: Bewertungen der Visualisierungsexperten im Vergleich zu den Bewertungen aus der Literaturrecherche. U = Bewertung aus der Umfrage, L = Bewertung aus der Literaturrecherche, U-L = Differenz.

die Bewertungsskala leicht nach oben verschoben wurde. Besonders deutlich ist die Abweichung bei der Streetmap Visualisierung, die in der Literaturrecherche mit 5 Punkten bewertet wurde, von den Experten jedoch nur 2,3 Punkte erhält. Anscheinend sind Streetmap Visualisierungen schwerer zu erfassen als in der Literaturrecherche angenommen.

Eine weitere Auffälligkeit zeigt sich bei der Skalierbarkeit von Sunburst, die in der Literaturrecherche mit 4 Punkten bewertet wurde, von den Experten jedoch nur 2,2 Punkte erhält. In der Praxis scheint bei großen Codebasen die Interpretierbarkeit von Sunburst Visualisierungen schlecht zu sein, als in der Literaturrecherche angenommen.

Trotzdem kommen die Ergebnisse der Literaturrecherche in weiten Teilen den Einschätzungen der Experten sehr nahe, was die Validität der Literaturbewertungen stützt. Eine Ausnahme bildet Streetmap, die von den Experten deutlich schlechter bewertet wurde als in der Literaturrecherche angenommen.

2.2.3 Zusätzliche Anmerkungen der Befragten

Zur ersten Frage, ob Software-Visualisierungen generell sinnvoll seien und warum, gaben mehrere Teilnehmende an, Software-Visualisierungen insgesamt als sehr sinnvoll und nützlich einzuschätzen. Sie könnten helfen, komplexe Inhalte verständlicher darzustellen und einen schnellen Überblick zu ermöglichen. Metriken sollten allerdings mit Bedacht und klarer Zielsetzung gewählt werden. Besonders in größeren Projekten tragen Visualisierungen dazu bei, Probleme sichtbar zu machen und die Kommunikation innerhalb der Entwicklerteams zu erleichtern, auch für Personen ohne Programmiererfahrung. Zudem wurde berichtet, dass sich Teambesprechungen effizienter gestalten ließen und die visuelle Komplexität reduziert werde.

Zur zweiten Frage, ob es konkrete Anmerkungen zu einzelnen Visualisierungen gebe, wurde allgemein kritisiert, dass Beschriftungen häufig zu klein, schwer lesbar und schwer zuzuordnen seien, was die Navigation erschwere und die schnelle Erfassung der Inhalte behindere. Zudem seien Details im Hintergrund schwer erkennbar. Besonders positiv hervorgehoben wurde das Circle Packing, da hier Unterschiede zwischen Ordnern und Dateien klar erkennbar würden. Negativ bewertet wurde hingegen die Streetmap, die als unübersichtlich und unleserlich beschrieben wurde. Darüber hinaus

Platzierung	Literaturrecherche	Szenarien	Direkte Bewertung
1	Treemap (4.0)	Treemap und Circular Treemap (3.9)	Treemap (3.8)
2	Code City und Streetmap (3.8)		Code City (3.4)
3		Code City (3.7)	Sunburst (3.0)
4	Sunburst (3.5)	Sunburst (3.1)	Circular Treemap (2.9)
5	Circular Treemap (3.2)	Streetmap (3.0)	Streetmap (2.6)

Tabelle 2.3: Platzierung der Visualisierungen in den drei Bewertungsansätzen. In Klammern die jeweilige Durchschnittsbewertung.

wurde darauf hingewiesen, dass die Treemap in einer 2D-Darstellung deutlich besser funktionieren würde als in 3D. Es wurde zudem betont, dass je nach Fragestellung unterschiedliche Visualisierungen besser geeignet seien.

Auf die dritte Frage, ob es generelle Kritik zu Aspekten gebe, die nicht abgefragt wurden, äußerten die Teilnehmenden Anmerkungen, die in der Umfrage nicht explizit adressiert waren. So wurde angemerkt, dass manche Visualisierungen nicht zu den jeweiligen Szenarien passten, was den Befragten bewusst verschwiegen wurde. Auch wenn diese Visualisierungen daraufhin schlecht bewertet wurden, sei dies positiv, da in der Praxis dadurch fehlerhafte Expertenanalysen identifiziert werden könnten. Zudem wurde der Wunsch nach zusätzlichen Beziehungsmetriken geäußert, da beispielsweise nicht erkannt worden sei, dass stets dieselbe Codebasis verwendet wurde. Generell bestand Interesse an weiterführenden und tiefergehenden Metriken, um die Aussagekraft der Visualisierungen zu erhöhen.

2.3 Fazit

Werden die drei Ansätze zur Bewertung der Visualisierungen miteinander verglichen und in eine Rangfolge gebracht, ergibt sich das in Tabelle 2.3 dargestellte Bild.

Auffällig ist, dass die Circular Treemap sowohl in der Literaturrecherche als auch bei den Experten schwach abschneidet, in den Szenarien jedoch zu den besten gehört. Zudem wurde die nicht passende Visualisierung der Circular Treemap vergleichsweise gut erkannt. An dieser Stelle deutet sich eine Diskrepanz zwischen den Bewertungskriterien und der wahrgenommenen Nützlichkeit in den Szenarien an. Möglicherweise können die vier Kriterien, um die Nachvollziehbarkeit von Visualisierungen direkt zu bewerten noch angepasst und verbessert werden, um die wahrgenommene Nützlichkeit der Visualisierungen zur Nachvollziehbarkeit von Expertenanalysen besser abzubilden.

Die Streetmap Visualisierung ist in den Szenarien und in der direkten Bewertung der Visualisierungsexperten am unbeliebtesten. In der Literaturrecherche schneidet sie besser ab, eventuell wurde dort die visuelle Komplexität unterschätzt. Die Sunburst-Visualisierung erreicht in allen drei Ansätzen mittlere Werte. Code City erreicht auch mittlere Werte, schneidet aber durchweg etwas besser ab als Sunburst. Treemap Visualisierungen erzielen in allen drei Ansätzen die besten Ergebnisse und werden zusätzlich gut als

nicht passend erkannt. Treemap Visualisierungen scheinen somit die beste Visualisierung für den in dieser Arbeit definierten Anwendungsfall zu sein.

FAZIT

In dieser arbeit konnte natürlich ein eine endliche Anzahl an Ansätzen und Ideen untersucht werden. In Zukunft ist bestimmt noch viel kreativität und Forschungspotential in diesem Bereich. Außerdem vielleicht sogar komplett neue Ansätze, die nicht auf der Stadt-Metapher basieren sondern vielleicht auf planeten oder anderen Metaphern, auch wenn zu vermuten ist, dass dabei die übersichlichekeit und einfachheit leiden könnte. Die in diser arbeit untersuchten Ansätze sind alle sehr einfach mit basic formen und daher übersichtlich.

Es ist natürlich klar, dass der treempa ansatz optmiert wurde und die anderen Ansätze nicht, weshalb der treemap ansatz besere ergebnisse liefert. Es bleibt zu erforschen, ob sich die anderen Ansätze auch so optimieren lassen, dass sie bessere Ergebnisse liefern.

Außerdem ist bei der literatur recherche noch interessant auch komplett über den tellerrand von software hinaus zu schauen und andere hierarchical vis zu betrachten, die nicht direkt mit software zu tun haben, um eventuell neue gut viz zu finden. Oder sogar ganz anders mit Kreativen ideen findungs ansätzen komplett neue Ansätze zu finden. Oder sich neue methaphern zu überlegen und nicht nur stadt, inseln, weltall,

Außerdem ist es interannt in der umfrage die tools den nutzern direkt zur verfügung zu stellen. In dieser arbeit wurden tools nur mit statischen bildern vorgestllt, um sich konkret nur auf die Visualisierung zu konzentrieren und faktoren wie nutzerfreundlichkeit, performance und interaktion nicht mit zu betrachten, wie bereits in problemstellung erwähnt. In zukunft wäre es zum beispiel interessant verschiedene interaktionsmöglichkeiten mit den layouts, wie zoomen, filtern, suchen, navigieren, etc zu untersuchen und zu vergleichen

Außerdem kann es passieren, dass die Beschriftung zu lang für die breite der Knotens ist, wodurch die Beschriftung entweder abgeschnitten wird oder über den Knoten hinauswächst."das angehen, mit zum beispiel rechteck breiter machen oder so, anders paltzieren, PIPAPO

Außerdem eventuell dynamische Beschriftung implementieren

Auch interessant wenn die struktur oder die metriken nicht auf file, sondern zum beispiel auf klassen basis sind. schauen, ob das einen unterschied macht.

Bewusst, dass gewisse wissenschaftliche standards nicht zu 100 Przent eingehalten wurden Zb. hat nur eine person am code gearbeitet, es wurde speziell der algorithmus so lange implementiert, bis die ergebnisse korrekt aussahen, das ist keine 100 prozentige garantie, dass zb die evaluation wirklich korrekt und fehlerfrei ist. interessant wäre nochmal eine arbeit, die den code überprüft. aber bei software engineering immer so, dass es bugs und

fehler gibt, nie auszuschließen. aber in dieser arbeit größte vorsicht und gewissenhaftigkeit - auch wenn das nicht wissenschaftlich ist. Deswegen stellen wir den kompletten quellcode zur verfügung, damit andere die Ergebnisse überprüfen können. Vorallem wichtig, dass man sich dessen bewusst ist, auch wenn man nichts dagegen tun kann

Gehe auf das Potenzial von nicht getesteten Ansätzen ein zb bubble tree-maps, könnte sehr cool sein

In der suche haben wir viele verschiedene Ansätze vorgestellt, die auch verschieden fokus haben, zum beispiel auf die darstellung von gebäuden, interessant auch das zu vergleichen

Es wäre interessant alle aspekte zu vergleich, also auch detail grad, farben,

Man könnte nochmal die Kriterien spezifisch testen

Es wäre interessant heraus zu finden, warum 1,05 als zielwert besser als 1 ist.

Noch mehr die MehrfacheBerechnung austesten

interessant wäre eine noch größere Umfrage

codecharta bias von experten?

Teil II

APPENDIX

DATEIEN

Alle Dateien sind auch digital unter diesem [Link \[Meh25\]](#) zu finden.

```
{
  "nodes": [
    {
      "name": "root",
      "type": "Folder",
      "attributes": {},
      "children": [
        {
          "name": "leaf 0",
          "type": "File",
          "attributes": {
            "rloc": 10,
            "functions": 5,
            "mcc": 100
          }
        },
        {
          "name": "small_parent_1",
          "type": "Folder",
          "attributes": {},
          "children": [
            {
              "name": "leaf 1",
              "type": "File",
              "attributes": {
                "rloc": 30,
                "functions": 100,
                "mcc": 100
              }
            },
            {
              "name": "small_parent_2",
              "type": "Folder",
              "attributes": {},
              "children": [
                {
                  "name": "leaf 2",
                  "type": "File",
                  "attributes": {
```

```

        "rloc": 30,
        "functions": 10,
        "mcc": 100
    }
},
{
    "name": "small_parent_3",
    "type": "Folder",
    "attributes": {},
    "children": [
        {
            "name": "leaf 3",
            "type": "File",
            "attributes": {
                "rloc": 30,
                "functions": 10,
                "mcc": 100
            }
        }
    ]
}
]
}
]
},
{
    "name": "huge_parent",
    "type": "Folder",
    "attributes": {},
    "children": [
        {
            "name": "huge leaf",
            "type": "File",
            "attributes": {
                "rloc": 3000,
                "functions": 100,
                "mcc": 100
            }
        }
    ]
}
]
}
]
}
]
}

```

Listing A.1: Händisch erzeugte map

```

{
  "nodes": [
    {
      "name": "root",
      "type": "Folder",
      "attributes": {},
      "children": [
        {
          "name": "leaf 0",
          "type": "File",
          "attributes": {
            "rloc": 10,
            "functions": 5,
            "mcc": 100
          }
        },
        {
          "name": "small_parent_1",
          "type": "Folder",
          "attributes": {},
          "children": [
            {
              "name": "leaf 1",
              "type": "File",
              "attributes": {
                "rloc": 30,
                "functions": 100,
                "mcc": 100
              }
            }
          ],
          "name": "small_parent_2",
          "type": "Folder",
          "attributes": {},
          "children": [
            {
              "name": "leaf 2",
              "type": "File",
              "attributes": {
                "rloc": 30,
                "functions": 10,
                "mcc": 100
              }
            }
          ],
          "name": "small_parent_3",

```

```

"type": "Folder",
"attributes": {},
"children": [
  {
    "name": "leaf 3",
    "type": "File",
    "attributes": {
      "rloc": 30,
      "functions": 10,
      "mcc": 100
    }
  },
  {
    "name": "small_parent_4",
    "type": "Folder",
    "attributes": {},
    "children": [
      {
        "name": "leaf 4",
        "type": "File",
        "attributes": {
          "rloc": 30,
          "functions": 10,
          "mcc": 100
        }
      }
    ],
    {
      "name": "small_parent_5",
      "type": "Folder",
      "attributes": {},
      "children": [
        {
          "name": "leaf 5",
          "type": "File",
          "attributes": {
            "rloc": 30,
            "functions": 10,
            "mcc": 100
          }
        }
      ],
      {
        "name": "small_parent_6",
        "type": "Folder",
        "attributes": {},
        "children": [
          {

```

```

    "name": "leaf 6",
    "type": "File",
    "attributes": {
        "rloc": 30,
        "functions": 10,
        "mcc": 100
    }
},
{
    "name": "small_parent_7",
    "type": "Folder",
    "attributes": {},
    "children": [
        {
            "name": "leaf 7",
            "type": "File",
            "attributes": {
                "rloc": 30,
                "functions": 10,
                "mcc": 100
            }
        }
    ],
    {
        "name": "small_parent_8",
        "type": "Folder",
        "attributes": {},
        "children": [
            {
                "name": "leaf 8",
                "type": "File",
                "attributes": {
                    "rloc": 30,
                    "functions": 10,
                    "mcc": 100
                }
            }
        ],
        {
            "name":
                "small_parent_9",
            "type": "Folder",
            "attributes": {},
            "children": [
                {
                    "name": "leaf 9",
                    "type": "File",
                    "attributes": {

```



```

{
  "link": "https://github.com/nvbn/thefuck",
  "description": "Magnificent app which corrects your
    previous console command."
},
{
  "link": "https://github.com/scikit-learn/scikit-learn",
  "description": "scikit-learn: machine learning in Python"
},
{
  "link": "https://github.com/ansible/ansible",
  "description": "Ansible is a radically simple IT
    automation platform that makes your applications and
    systems easier to deploy and maintain. Automate
    everything from code deployment to network
    configuration to cloud management, in a language that
    approaches plain English, using SSH, with no agents
    to install on remote systems.
    https://docs.ansible.com."
},
{
  "link": "https://github.com/CorentinJ/Real-Time-Voice-Cloning",
  "description": "Clone a voice in 5 seconds to generate
    arbitrary speech in real-time"
},
{
  "link": "https://github.com/apache/airflow",
  "description": "Apache Airflow - A platform to
    programmatically author, schedule, and monitor
    workflows"
},
{
  "link": "https://github.com/harry0703/MoneyPrinterTurbo",
  "description": "Generate short videos with one click using
    AI LLM."
},
{
  "link": "https://github.com/freqtrade/freqtrade",
  "description": "Free, open source crypto trading bot"
},
{
  "link": "https://github.com/streamlit/streamlit",
  "description": "Streamlit A faster way to build and share
    data apps."
},
{

```

```

        "link": "https://github.com/mlflow/mlflow",
        "description": "The open source developer platform to
            build AI/LLM applications and models with confidence.
            Enhance your AI applications with end-to-end
            tracking, observability, and evaluations, all in one
            integrated platform."
    },
    {
        "link": "https://github.com/commaai/openpilot",
        "description": "openpilot is an operating system for
            robotics. Currently, it upgrades the driver
            assistance system on 300+ supported cars."
    },
    {
        "link": "https://github.com/pytorch/pytorch",
        "description": "Tensors and Dynamic neural networks in
            Python with strong GPU acceleration"
    },
    {
        "link": "https://github.com/pytorch/vision",
        "description": "Datasets, Transforms and Models specific
            to Computer Vision"
    },
    {
        "link": "https://github.com/frappe/erpnext",
        "description": "Free and Open Source Enterprise Resource
            Planning (ERP)"
    },
    {
        "link": "https://github.com/StevenBlack/hosts",
        "description": "Consolidating and extending hosts files
            from several well-curated sources. Optionally pick
            extensions for porn, social media, and other
            categories."
    },
    {
        "link": "https://github.com/macrozheng/mall",
        "description": "mallBoot+MyBatis"
    },
    {
        "link": "https://github.com/zxing/zxing",
        "description": "ZXing (Crossingbarcode scanning library
            for Java, Android"
    },
    {
        "link": "https://github.com/scwang90/SmartRefreshLayout",

```

```

    "description": "Footer"
  },
  {
    "link": "https://github.com/oracle/graal",
    "description": "GraalVM compiles Java applications into
      native executables that start instantly, scale fast,
      and use fewer compute resources "
  },
  {
    "link": "https://github.com/keycloak/keycloak",
    "description": "Open Source Identity and Access Management
      For Modern Applications and Services"
  },
  {
    "link": "https://github.com/doocs/leetcode",
    "description": "solutions in any programming language |
      LeetCodeOffer2 6 "
  },
  {
    "link": "https://github.com/neo4j/neo4j",
    "description": "Graphs for Everyone"
  },
  {
    "link": "https://github.com/deeplearning4j/deeplearning4j",
    "description": "Suite of tools for deploying and training
      deep learning models using the JVM. Highlights
      include model import for keras, tensorflow, and
      onnx/pytorch, a modular and tiny c++ library for
      running math code and a java based math library on
      top of the core c++ library. Also includes samediff:
      a pytorch/tensorflow like library for running deep
      learn..."
  },
  {
    "link": "https://github.com/OpenAPITools/openapi-generator",
    "description": "OpenAPI Generator allows generation of API
      client libraries (SDK generation), server stubs,
      documentation and configuration automatically given
      an OpenAPI Spec (v2, v3)"
  },
  {
    "link": "https://github.com/SonarSource/sonarqube",
    "description": "Continuous Inspection"
  },
  {
    "link": "https://github.com/libgdx/libgdx",

```

```

        "description": "Desktop/Android/HTML5/iOS Java game
                        development framework"
    },
    {
        "link": "https://github.com/flyway/flyway",
        "description": "Flyway by Redgate Database Migrations Made
                        Easy."
    },
    {
        "link": "https://github.com/openjdk/jdk",
        "description": "JDK main-line development
                        https://openjdk.org/projects/jdk"
    },
    {
        "link": "https://github.com/arduino/Arduino",
        "description": "Arduino IDE 1.x"
    },
    {
        "link": "https://github.com/sveltejs/svelte",
        "description": "web development for the rest of us"
    },
    {
        "link": "https://github.com/danielmiessler/Fabric",
        "description": "Fabric is an open-source framework for
                        augmenting humans using AI. It provides a modular
                        system for solving specific problems using a
                        crowdsourced set of AI prompts that can be used
                        anywhere."
    },
    {
        "link": "https://github.com/open-webui/open-webui",
        "description": "User-friendly AI Interface (Supports
                        Ollama, OpenAI API, ...)"
    },
    {
        "link": "https://github.com/atom/atom",
        "description": ":atom: The hackable text editor"
    },
    {
        "link": "https://github.com/phaserjs/phaser",
        "description": "Phaser is a fun, free and fast 2D game
                        framework for making HTML5 games for desktop and
                        mobile web browsers, supporting Canvas and WebGL
                        rendering."
    },
    {

```

```

    "link": "https://github.com/swagger-api/swagger-ui",
    "description": "Swagger UI is a collection of HTML,
        JavaScript, and CSS assets that dynamically generate
        beautiful documentation from a Swagger-compliant API."
  },
  {
    "link": "https://github.com/handsontable/handsontable",
    "description": "JavaScript Data Grid / Data Table with a
        Spreadsheet Look & Feel. Works with React, Angular,
        and Vue. Supported by the Handsontable team "
  },
  {
    "link": "https://github.com/zhaoolee/ChromeAppHeroes",
    "description": " ChromePluginHeroes, Write a Chinese
        manual for the excellent Chrome plugin, let the
        Chrome plugin heroes benefit the human~ 01"
  },
  {
    "link": "https://github.com/aframevr/aframe",
    "description": ":a: Web framework for building virtual
        reality experiences."
  },
  {
    "link": "https://github.com/4ian/GDevelop",
    "description": "Open-source, cross-platform
        2D/3D/multiplayer game engine designed for everyone."
  },
  {
    "link": "https://github.com/facebook/react",
    "description": "The library for web and native user
        interfaces."
  },
  {
    "link": "https://github.com/gatsbyjs/gatsby",
    "description": "The best React-based framework with
        performance, scalability and security built in."
  },
  {
    "link": "https://github.com/nodejs/node",
    "description": "Node.js JavaScript runtime "
  },
  {
    "link": "https://github.com/ToolJet/ToolJet",
    "description": "Low-code platform for building business
        applications. Connect to databases, cloud storages,
        GraphQL, API endpoints, Airtable, Google sheets,

```

```

        OpenAI, etc and build apps using drag and drop
        application builder. Built using
        JavaScript/TypeScript. "
    },
    {
        "link":"https://github.com/romkatv/powerlevel10k",
        "description":"A Zsh theme"
    },
    {
        "link":"https://github.com/testssl/testssl.sh",
        "description":"Testing TLS/SSL encryption anywhere on any
        port "
    },
    {
        "link":"https://github.com/linux-surface/linux-surface",
        "description":"Linux Kernel for Surface Devices"
    },
    {
        "link":"https://github.com/ONLYOFFICE/DocumentServer",
        "description":"ONLYOFFICE Docs is a free collaborative
        online office suite comprising viewers and editors
        for texts, spreadsheets and presentations, forms and
        PDF, fully compatible with Office Open XML formats:
        .docx, .xlsx, .pptx and enabling collaborative
        editing in real time."
    },
    {
        "link":"https://github.com/facebookarchive/caffe2",
        "description":"Caffe2 is a lightweight, modular, and
        scalable deep learning framework."
    },
    {
        "link":"https://github.com/mitchellkrogza/
        nginx-ultimate-bad-bot-blocker",
        "description":"Nginx Block Bad Bots, Spam Referrer
        Blocker, Vulnerability Scanners, User-Agents,
        Malware, Adware, Ransomware, Malicious Sites, with
        anti-DDOS, Wordpress Theme Detector Blocking and
        Fail2Ban Jail for Repeat Offenders"
    },
    {
        "link":"https://github.com/dtcooper/raspotify",
        "description":"A Spotify Connect client that mostly Just
        Works"
    },
    {

```

```

    "link": "https://github.com/armbian/build",
    "description": "Armbian Linux build framework generates
                    custom Debian or Ubuntu image for x86, aarch64,
                    riscv64 & armhf"
  },
  {
    "link": "https://github.com/filsv/iOSDeviceSupport",
    "description": "Xcode iPhoneOS (iOS) DeviceSupport files
                    (6.0 - 17.0)"
  },
  {
    "link": "https://github.com/prasanthrangan/hyprdots",
    "description": "// Aesthetic, dynamic and minimal dots for
                    Arch hyprland"
  },
  {
    "link": "https://github.com/hoverbike1/TOTK-Mods-collection",
    "description": "Mod repo for Tears of The Kingdom (TOTK)
                    for Switch and Switch Emulation"
  },
  {
    "link": "https://github.com/HyDE-Project/HyDE",
    "description": "HyDE, your Development Environment "
  },
  {
    "link": "https://github.com/yonggekkk/x-ui-yg",
    "description": "x-uiXhttpargoPsiphonVPN30sing-boxclash-meta"
  },
  {
    "link": "https://github.com/fuzhengwei/CodeGuide",
    "description": ":books: Java Java()"
  },
  {
    "link": "https://github.com/pbatard/rufus",
    "description": "The Reliable USB Formatting Utility"
  },
  {
    "link": "https://github.com/mpv-player/mpv",
    "description": "Command line media player"
  },
  {
    "link": "https://github.com/jart/cosmopolitan",
    "description": "build-once run-anywhere c library"
  },
  {
    "link": "https://github.com/coolsnowwolf/lede",

```

```

    "description": "Lean's LEDE source"
  },
  {
    "link": "https://github.com/wazuh/wazuh",
    "description": "Wazuh - The Open Source Security Platform.
      Unified XDR and SIEM protection for endpoints and
      cloud workloads."
  },
  {
    "link": "https://github.com/videolan/vlc",
    "description": "VLC media player - All pull requests are
      ignored, please use MRs on
      https://code.videolan.org/videolan/vlc"
  },
  {
    "link": "https://github.com/reactos/reactos",
    "description": "A free Windows-compatible Operating System"
  },
  {
    "link": "https://github.com/TelegramMessenger/Telegram-iOS",
    "description": "Telegram-iOS"
  },
  {
    "link": "https://github.com/yugabyte/yugabyte-db",
    "description": "YugabyteDB - the cloud native distributed
      SQL database for mission-critical applications."
  },
  {
    "link": "https://github.com/OnionUI/Onion",
    "description": "OS overhaul for Miyoo Mini and Mini+"
  },
  {
    "link": "https://github.com/wine-mirror/wine",
    "description": null
  },
  {
    "link": "https://github.com/wireshark/wireshark",
    "description": "Read-only mirror of Wireshark's Git
      repository at https://gitlab.com/wireshark/wireshark.
      GitHub won't let us disable pull requests. THEY WILL
      BE IGNORED HERE Upload them at GitLab instead."
  },
  {
    "link": "https://github.com/darktable-org/darktable",
    "description": "darktable is an open source photography
      workflow application and raw developer"
  }

```

```
    },  
    {  
      "link": "https://github.com/hanwckf/rt-n56u",  
      "description": "Padavan"  
    }  
  ]
```

Listing A.3: Die zur Analyse ausgewählten GitHub-Repositories

LITERATUR

- [Sof] [Online; besucht 22-Juni-2025]. 2022. URL: <https://octoverse.github.com/2022/state-of-open-source>.
- [Sur] [Online; besucht 4-September-2025]. 2023. URL: <https://surveysmars.com/de/>.
- [Cod] [Online; besucht 10-August-2025]. 2025. URL: <https://github.com/quantifiedcode/code-is-beautiful>.
- [Eva] [Online; besucht 23-Juni-2025]. 2025. URL: <https://github.com/EvanLi/Github-Ranking?tab=readme-ov-file#most-stars>.
- [Gita] [Online; besucht 12-September-2025]. 2025. URL: <https://github.com/facebook/react>.
- [Gitb] [Online; besucht 12-September-2025]. 2025. URL: <https://github.com/QwenLM/Qwen3>.
- [D3t] [Online; besucht 19-Mai-2025]. 2025. URL: <https://d3js.org/d3-hierarchy/treemap>.
- [Bos23] Mike Bostock. *Observable*. [Online; besucht 13-September-2025]. 2023. URL: <https://observablehq.com/@d3/nested-treemap>.
- [BHVWoo] Mark Bruls, Kees Huizing und Jarke J Van Wijk. "Squarified treemaps". In: *Data Visualization 2000: Proceedings of the Joint EUROGRAPHICS and IEEE TCVG Symposium on Visualization in Amsterdam, The Netherlands*. Springer. 2000, S. 33–42.
- [CAC20] Andrea Capiluppi, Nemitari Ajenka und Steve Counsell. "The effect of multiple developers on structural attributes: A Study based on java software". In: *Journal of Systems and Software* 167 (2020), S. 110593. ISSN: 0164-1212. DOI: <https://doi.org/10.1016/j.jss.2020.110593>. URL: <https://www.sciencedirect.com/science/article/pii/S016412122030073X>.
- [ETB11] Ural Erdemir, Umut Tekin und Feza Buzluca. "E-Quality: A graph based object oriented software quality visualization tool". In: *2011 6th International Workshop on Visualizing Software for Understanding and Analysis (VISSOFT)*. 2011, S. 1–8. DOI: [10.1109/VISSOFT.2011.6069454](https://doi.org/10.1109/VISSOFT.2011.6069454).
- [Gmba] Generative AI Seerene GmbH. *Sereene Website*. [Online; besucht 29-Juli-2025]. URL: <https://www.seerene.com/de/>.
- [Gmbb] MaibornWolff GmbH. *CodeCharta GitHub*. [Online; besucht 04-August-2025]. URL: <https://github.com/MaibornWolff/codecharta>.

- [Gmbc] MaibornWolff GmbH. *CodeCharta Web Demo*. [Online; besucht 29-Juli-2025]. URL: <https://codecharta.com/visualization/app/index.html>.
- [Gmbd] MaibornWolff GmbH. *CodeCharta Wiki Analysis*. [Online; besucht 31-Juli-2025]. URL: <https://codecharta.com/docs/analysis/codecharta-shell>.
- [Joh93] Brian Scott Johnson. *Treemaps: Visualizing hierarchical and categorical data*. University of Maryland, College Park, 1993.
- [JS91] Brian Johnson und Ben Shneiderman. *Tree-maps: A space filling approach to the visualization of hierarchical information structures*. Techn. Ber. UM Computer Science Department; CS-TR-2657, 1991.
- [LFo8] Hao Lü und James Fogarty. "Cascaded treemaps: examining the visibility and stability of structure in treemaps". In: *Proceedings of graphics interface 2008*. 2008, S. 259–266.
- [Meh25] Benedikt Mehl. *Anhang zu Masterarbeit*. 2025. URL: <https://github.com/BenediktMehl/master-thesis/tree/main/anhang>.
- [PSE04] J.W. Paulson, G. Succi und A. Eberlein. "An empirical study of open-source and closed-source software products". In: *IEEE Transactions on Software Engineering* 30.4 (2004), S. 246–256. DOI: [10.1109/TSE.2004.1274044](https://doi.org/10.1109/TSE.2004.1274044).
- [Rag+05] S. Raghunathan, A. Prasad, B.K. Mishra und Hsihui Chang. "Open source versus closed source: software quality in monopoly and competitive markets". In: *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans* 35.6 (2005), S. 903–918. DOI: [10.1109/TSMCA.2005.853493](https://doi.org/10.1109/TSMCA.2005.853493).
- [Saj+14] Hitesh Sajnani, Vaibhav Saini, Joel Ossher und Cristina V. Lopes. "Is Popularity a Measure of Quality? An Analysis of Maven Components". In: *2014 IEEE International Conference on Software Maintenance and Evolution*. 2014, S. 231–240. DOI: [10.1109/ICSME.2014.45](https://doi.org/10.1109/ICSME.2014.45).
- [SLD20] Willy Scheibel, Daniel Limberger und Jürgen Döllner. "Survey of treemap layout algorithms". In: *Proceedings of the 13th international symposium on visual information communication and interaction*. 2020, S. 1–9.
- [SWo1] Ben Shneiderman und Martin Wattenberg. "Ordered treemap layouts". In: *IEEE Symposium on Information Visualization, 2001. INFOVIS 2001*. IEEE. 2001, S. 73–78.
- [WLo7] Richard Wettel und Michele Lanza. "Visualizing Software Systems as Cities". In: *2007 4th IEEE International Workshop on Visualizing Software for Understanding and Analysis*. 2007, S. 92–99. DOI: [10.1109/VISSOF.2007.4290706](https://doi.org/10.1109/VISSOF.2007.4290706).